

BÀI 3: JAVASCRIPT CƠ BẢN TRÊN TRÌNH DUYỆT

1. Mục tiêu của bài:

- Hiểu được vai trò của JavaScript trong việc tạo tương tác trên trang web.
- Sử dụng được JavaScript cơ bản để xử lý sự kiện và thay đổi nội dung trang web.
- Áp dụng được JavaScript vào một số chức năng đơn giản như kiểm tra dữ liệu, tạo hiệu ứng giao diện.

2. Nội dung bài:

2.1. Kiến thức cơ bản về JavaScript

JavaScript là ngôn ngữ lập trình được sử dụng rộng rãi để điều khiển hành vi của trang web. Nếu HTML đảm nhiệm vai trò tạo cấu trúc nội dung, CSS phụ trách trình bày giao diện, thì JavaScript chịu trách nhiệm xử lý tương tác, tính toán dữ liệu, kiểm tra điều kiện, phản hồi sự kiện và cập nhật nội dung hiển thị theo thời gian thực. Trong mô đun nhập môn thiết kế web, việc nắm chắc kiến thức JavaScript cơ bản có ý nghĩa nền tảng vì gần như mọi ứng dụng web đều cần đến biến, kiểu dữ liệu, câu lệnh điều khiển và cách gắn mã lệnh vào trang HTML. Ở giai đoạn đầu, chúng ta chưa đi sâu vào các thư viện hay framework mà cần hiểu đúng phần lõi của ngôn ngữ. Khi hiểu đúng các khái niệm cơ sở này, sinh viên sẽ dễ dàng đọc mã, sửa lỗi và phát triển các chức năng phức tạp hơn trong các bài học tiếp theo.

2.1.1. Khai báo biến, kiểu dữ liệu cơ bản, toán tử số học, so sánh, logic

a. Khái niệm biến trong JavaScript

Biến là vùng nhớ có tên dùng để lưu trữ dữ liệu trong quá trình chương trình thực thi. Dữ liệu này có thể thay đổi hoặc giữ nguyên tùy theo cách khai báo. Khai báo biến trong JavaScript chúng ta dùng:

- `let`: dùng khi giá trị của biến có thể thay đổi.
- `const`: dùng khi không muốn gán lại giá trị cho biến sau khi khởi tạo.
- `var`: cú pháp cũ, vẫn tồn tại nhưng không ưu tiên trong thực hành cơ bản.

Trong JavaScript hiện đại, chúng ta thường sử dụng `let` và `const`, hạn chế dùng `var` vì `var` có cơ chế phạm vi hoạt động dễ gây nhầm lẫn cho người mới học. Ví dụ khai báo biến

```
let studentName = "An";  
let age = 20;  
const schoolName = "Cao đẳng Kỹ thuật";
```

Trong ví dụ trên:

- `studentName` lưu chuỗi ký tự.
- `age` lưu số nguyên.
- `schoolName` là hằng số, sau khi gán giá trị thì không nên gán lại.

Nếu viết:

```
age = 21;
```

thì hợp lệ vì `age` được khai báo bằng `let`.

Nếu viết:

```
schoolName = "Trường mới";
```

thì chương trình sẽ phát sinh lỗi vì `schoolName` được khai báo bằng `const`.

c. Kiểu dữ liệu cơ bản

JavaScript là ngôn ngữ định kiểu động, nghĩa là chúng ta không cần khai báo trước kiểu dữ liệu của biến như trong C hoặc Java. Kiểu dữ liệu được xác định dựa trên giá trị đang được gán. Bảng dưới đây tóm tắt một số kiểu dữ liệu cơ bản thường dùng

Kiểu dữ liệu	Ý nghĩa	Ví dụ
Number	Biểu diễn số nguyên hoặc số thực	10, 3.14
String	Biểu diễn chuỗi ký tự	"Hello", "Xin chào"
Boolean	Biểu diễn giá trị đúng hoặc sai	true, false
Undefined	Biến đã khai báo nhưng chưa gán giá trị	let x;
Null	Biểu diễn giá trị rỗng có chủ đích	let data = null;

Bảng 4.1: Các kiểu dữ liệu cơ bản trong JavaScript

Ví dụ:

```
let score = 8.5;           // Number
let fullName = "Nguyen Van A"; // String
let isPassed = true;       // Boolean
let address;               // Undefined
let selectedItem = null;    // Null
```

Để kiểm tra kiểu dữ liệu, chúng ta có thể dùng toán tử `typeof`

```
console.log(typeof score);    // number
console.log(typeof fullName); // string
console.log(typeof isPassed); // boolean
console.log(typeof address);  // undefined
```

d. Toán tử số học

Toán tử số học được dùng để thực hiện phép tính trên các toán hạng kiểu số. Đây là nội dung rất quan trọng vì nhiều chương trình sau này đều cần tính tổng, hiệu, trung bình, phần trăm hoặc đếm số lần lặp. Một số toán tử số học cơ bản gồm

Toán tử	Ý nghĩa	Ví dụ	Kết quả
+	Cộng	5 + 2	7
-	Trừ	5 - 2	3
*	Nhân	5 * 2	10
/	Chia	5 / 2	2.5
%	Chia lấy dư	5 % 2	1
**	Lũy thừa	2 ** 3	8

Bảng 4.2: Các toán tử số học thường dùng

Ví dụ:

```
let a = 10;
let b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Ngoài ra còn có các phép tăng giảm nhanh:

```
let count = 5;
count++;
console.log(count); // 6
count--;
console.log(count); // 5
```

e. Toán tử so sánh

Toán tử so sánh được dùng để kiểm tra quan hệ giữa hai giá trị. Kết quả trả về luôn là true hoặc false. Đây là tiền đề để xây dựng các câu lệnh điều kiện. Toán tử so sánh bao gồm:

Toán tử	Ý nghĩa
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
==	Bằng về giá trị
===	Bằng nghiêm ngặt cả giá trị và kiểu dữ liệu
!=	Khác về giá trị
!==	Khác nghiêm ngặt

Bảng 4.3: Các toán tử so sánh

Ví dụ:

```
let x = 5;
let y = "5";
console.log(x == y);    // true
console.log(x === y);   // false
console.log(x != y);    // false
console.log(x !== y);   // true
```

Trong thực hành, chúng ta nên ưu tiên dùng === và !== để tránh các trường hợp ép kiểu ngầm làm sai lệch kết quả.

f. Toán tử logic

Toán tử logic dùng để kết hợp nhiều điều kiện hoặc đảo ngược giá trị logic của một điều kiện.

Toán tử	Ý nghĩa	Ví dụ
&&	Và	a > 0 && b > 0
	Hoặc	a > 0 b > 0
!	Phủ định	!(a > 0)

Bảng 4.4: Các toán tử logic

Ví dụ:

```
let age = 20;
let hasCard = true;
```

```
console.log(age >= 18 && hasCard); // true
console.log(age < 18 || hasCard);  // true
console.log(!hasCard);              // false
```

2.1.2. Câu lệnh rẽ nhánh và lặp trong JavaScript.

Trong lập trình, chương trình không chỉ chạy tuần tự từ trên xuống dưới mà thường phải đưa ra quyết định hoặc thực hiện lặp đi lặp lại một nhóm lệnh. Vì vậy, chúng ta cần nắm được hai nhóm cấu trúc điều khiển cơ bản là rẽ nhánh và lặp.

a. Câu lệnh rẽ nhánh

Câu lệnh rẽ nhánh cho phép chương trình thực hiện các khối lệnh khác nhau tùy theo điều kiện.

Cấu trúc if

```
if (điều_kiện) {
    // khối lệnh được thực hiện nếu điều kiện đúng
}
```

Ví dụ:

```
let score = 8;
if (score >= 5) {
    console.log("Đạt");
}
```

Nếu score >= 5 là true, chương trình in ra "Đạt".

Cấu trúc if...else

```
if (điều_kiện) {
    // khối lệnh khi điều kiện đúng
} else {
    // khối lệnh khi điều kiện sai
}
```

Ví dụ:

```
let score = 4;
if (score >= 5) {
    console.log("Đạt");
} else {
    console.log("Chưa đạt");
}
```

Cấu trúc if...else if...else

Cấu trúc này dùng khi cần phân loại nhiều trường hợp.

```
let score = 8.7;
if (score >= 8.5) {
    console.log("Giỏi");
} else if (score >= 7) {
    console.log("Khá");
} else if (score >= 5) {
    console.log("Trung bình");
} else {
    console.log("Yếu");
}
```

Ở đây, chương trình kiểm tra lần lượt từ trên xuống dưới. Khi gặp điều kiện đúng đầu tiên, nó thực hiện khối lệnh tương ứng rồi dừng chuỗi kiểm tra.

b. Câu lệnh switch

switch phù hợp khi so sánh một biến với nhiều giá trị cụ thể.

```
let day = 3;
switch (day) {
    case 2:
        console.log("Thứ hai");
        break;
    case 3:
        console.log("Thứ ba");
        break;
    case 4:
        console.log("Thứ tư");
        break;
    default:
        console.log("Giá trị không hợp lệ");
}
```

Trong cấu trúc này:

- case đại diện cho từng trường hợp cụ thể.
- break giúp kết thúc nhánh hiện tại, tránh chạy tiếp sang các nhánh sau.
- default là nhánh mặc định nếu không khớp trường hợp nào.

c. Khái niệm vòng lặp

Vòng lặp dùng để thực hiện lặp lại một khối lệnh nhiều lần. Đây là công cụ quan trọng để xử lý danh sách dữ liệu, in ra nhiều kết quả, tính tổng nhiều giá trị hoặc duyệt qua tập hợp.

Vòng lặp for

Vòng lặp for thường dùng khi biết trước số lần lặp.

```
for (khởi_tạo; điều_kiện; cập_nhật) {  
    // khối lệnh lặp  
}
```

Ví dụ:

```
for (let i = 1; i <= 5; i++) {  
    console.log("Lần lặp thứ: " + i);  
}
```

Phân tích:

- let i = 1: khởi tạo biến đếm.
- i <= 5: điều kiện để tiếp tục lặp.
- i++: sau mỗi lần lặp, tăng i thêm 1.

Vòng lặp while

while thường dùng khi số lần lặp chưa biết trước, chương trình chỉ tiếp tục khi điều kiện còn đúng.

```
let i = 1;  
while (i <= 5) {  
    console.log("Giá trị i = " + i);  
    i++;  
}
```

Nếu quên câu lệnh i++, vòng lặp có thể chạy vô hạn. Đây là lỗi rất phổ biến với người mới học.

Vòng lặp do...while

do...while luôn thực hiện khối lệnh ít nhất một lần rồi mới kiểm tra điều kiện.

```
let i = 1;  
do {  
    console.log("Giá trị i = " + i);  
    i++;  
} while (i <= 5);
```

Bảng so sánh nhanh các cấu trúc điều khiển

Cấu trúc	Mục đích	Khi nào nên dùng
if	Kiểm tra một điều kiện	Khi chỉ có một nhánh cần xử lý

if...else	Chọn giữa hai nhánh	Khi có đúng hai khả năng
if...else if...else	Phân loại nhiều trường hợp	Khi điều kiện có nhiều mức
switch	So sánh với nhiều giá trị cụ thể	Khi một biến có nhiều giá trị rời rạc
for	Lặp với số lần xác định	Khi đã biết hoặc dễ xác định số vòng
while	Lặp khi điều kiện còn đúng	Khi phụ thuộc trạng thái biến
do...while	Lặp ít nhất một lần	Khi cần chạy trước rồi mới kiểm tra

Bảng 4.5: So sánh nhanh các cấu trúc điều khiển

Lỗi thường gặp khi dùng cấu trúc điều khiển:

Lỗi	Nguyên nhân	Hướng xử lý
Quên dấu ngoặc nhọn {}	Viết cấu trúc lệnh chưa hoàn chỉnh	Kiểm tra cặp ngoặc mở và đóng
Dùng = thay vì == hoặc ===	Nhầm giữa gán và so sánh	Phân biệt rõ phép gán với phép kiểm tra
Vòng lặp vô hạn	Quên cập nhật biến lặp	Luôn kiểm tra phần cập nhật
Điều kiện sai logic	Kết hợp &&, , ! chưa đúng	Tách điều kiện thành từng phần để kiểm tra
Thiếu break trong switch	Không hiểu luồng thực thi của switch	Thêm break sau mỗi case khi cần

Bảng 4.6: Lỗi thường gặp khi dùng cấu trúc điều khiển

2.1.3. Cách chèn JavaScript vào trang HTML (inline, internal, external)

Để JavaScript hoạt động trên trang web, chúng ta cần biết cách đưa mã JavaScript vào tài liệu HTML. Có ba cách cơ bản là inline, internal và external. Mỗi cách có phạm vi sử dụng khác nhau, và trong giáo trình này chúng ta cần chỉ ra rõ ưu điểm, hạn chế cũng như bối cảnh áp dụng.

a. Chèn JavaScript theo kiểu inline

Inline nghĩa là mã JavaScript được viết trực tiếp ngay trong thuộc tính của thẻ HTML, thường dùng với các sự kiện như onclick, onchange, onmouseover. Ví dụ:

```
<button onclick="alert('Xin chào JavaScript')">Nhấn vào  
đây</button>
```

Phân tích:

- Khi người dùng nhấn nút, đoạn mã trong onclick sẽ được thực thi.
- Cách này giúp minh họa nhanh tương tác đơn giản.
- Tuy nhiên, nếu lạm dụng thì mã HTML sẽ trở nên rối, khó bảo trì và khó tách biệt nội dung với xử lý nghiệp vụ.

Vì vậy, inline chỉ nên dùng để minh họa khái niệm ban đầu hoặc các ví dụ cực ngắn.

b. Chèn JavaScript theo kiểu internal

Internal nghĩa là viết mã JavaScript trong cặp thẻ <script> ngay bên trong file HTML. Ví dụ:

```
<!DOCTYPE html>  
<html lang="vi">  
<head>  
  <meta charset="UTF-8">  
  <title>Ví dụ Internal JavaScript</title>  
</head>  
<body>  
  <h1>Học JavaScript</h1>  
  <script>  
    let message = "Chào mừng đến với JavaScript";  
    console.log(message);  
  </script>  
</body>  
</html>
```

Phân tích:

- Trình duyệt đọc file HTML, gặp thẻ <script> thì thực thi mã bên trong.
- Cách này phù hợp cho các ví dụ nhỏ, một trang đơn giản hoặc bài thực hành đầu tiên.
- Khi số lượng mã tăng lên, việc đặt toàn bộ JavaScript trong HTML sẽ gây khó đọc và khó tái sử dụng.

c. Chèn JavaScript theo kiểu external

External nghĩa là đặt mã JavaScript vào một file riêng, thường có phần mở rộng .js, sau đó liên kết file này với HTML bằng thẻ <script src="..."></script>. Ví dụ file script.js:

```
let title = "Bài học JavaScript cơ bản";
console.log(title);
function sayHello() {
    alert("Xin chào từ file JavaScript bên ngoài");
}
```

Ví dụ file index.html:

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <title>External JavaScript</title>
</head>
<body>
    <h1>Liên kết file JavaScript</h1>
    <button onclick="sayHello()">Gửi lời chào</button>

    <script src="script.js"></script>
</body>
</html>
```

Phân tích:

- File HTML chịu trách nhiệm trình bày giao diện.
- File script.js chịu trách nhiệm xử lý logic.
- Đây là cách làm chuẩn trong phát triển web thực tế vì dễ bảo trì, dễ mở rộng, tái sử dụng tốt và giúp mã nguồn rõ ràng hơn.

d. Vị trí đặt thẻ <script>

Thẻ <script> có thể đặt trong phần <head> hoặc trước thẻ đóng </body>. Trong học phần nhập môn, chúng ta nên ưu tiên đặt gần cuối phần <body> để HTML được tải trước, sau đó JavaScript mới chạy. Cách này giúp hạn chế lỗi do JavaScript truy cập phần tử HTML khi phần tử đó chưa được tạo xong. Ví dụ:

```
<body>
    <h1 id="title">JavaScript cơ bản</h1>
    <script>
        document.getElementById("title").innerText = "Nội dung
```

```
đã được thay đổi";  
</script>  
</body>
```

Nếu đoạn script này đặt quá sớm trong <head> mà không có xử lý chờ tải xong trang, có thể phát sinh lỗi vì phần tử id="title" chưa tồn tại tại thời điểm script chạy.

Bảng so sánh ba cách chèn JavaScript

Cách chèn	Mô tả	Ưu điểm	Hạn chế	Khuyến nghị sử dụng
Inline	Viết trực tiếp trong thuộc tính HTML	Nhanh, dễ minh họa	Rối mã HTML, khó bảo trì	Chỉ dùng cho ví dụ rất ngắn
Internal	Viết trong thẻ <script> của file HTML	Dễ thử nghiệm, tiện cho bài nhỏ	Khó tái sử dụng khi dự án lớn	Dùng cho bài thực hành đơn giản
External	Viết trong file .js riêng	Tách biệt giao diện và logic, dễ quản lý	Cần thêm file liên kết	Nên dùng trong thực hành chuẩn

Bảng 4.7: So sánh ba cách chèn JavaScript vào HTML

Kế tiếp, chúng ta sẽ thực hiện một số bài thực hành để làm quen các khái niệm đã tìm hiểu trên.

Bài thực hành 1: Ôn nhanh biến, kiểu dữ liệu, toán tử

Bước 1. Tạo thư mục dự án

Tạo một thư mục tên js-basic-lesson.

Bước 2. Mở thư mục bằng VS Code

Trong VS Code, chọn **File > Open Folder** và mở thư mục vừa tạo.

Bước 3. Tạo file index.html

```
<!DOCTYPE html>  
<html lang="vi">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>JavaScript cơ bản</title>
```

```
</head>
<body>
  <h1>Thực hành JavaScript cơ bản</h1>
  <p>Mở Console để xem kết quả.</p>
  <script>
    let studentName = "Lan";
    let age = 19;
    let score = 8.5;
    let isPassed = score >= 5;
    console.log("Tên sinh viên:", studentName);
    console.log("Tuổi:", age);
    console.log("Điểm:", score);
    console.log("Đạt:", isPassed);
    console.log("Kiểu của studentName:", typeof
studentName);
    console.log("Kiểu của age:", typeof age);
    console.log("Kiểu của isPassed:", typeof isPassed);
    let a = 10;
    let b = 4;
    console.log("a + b =", a + b);
    console.log("a - b =", a - b);
    console.log("a * b =", a * b);
    console.log("a / b =", a / b);
    console.log("a % b =", a % b);
  </script>
</body>
</html>
```

Bước 4. Chạy file HTML

Có thể mở trực tiếp bằng trình duyệt hoặc dùng tiện ích **Live Server** trong VS Code.

Bước 5. Mở Console

Nhấn F12 hoặc chuột phải chọn **Inspect**, sau đó chọn thẻ **Console** để xem kết quả.

Giải thích mã

- Các biến studentName, age, score, isPassed minh họa các kiểu dữ liệu cơ bản.

- typeof giúp quan sát kiểu dữ liệu.
- Các phép toán với a và b minh họa nhóm toán tử số học.
- Cách này đặc biệt phù hợp ở buổi học đầu vì kết quả hiện ra rõ ràng, dễ quan sát.

Bài thực hành 2: Rẽ nhánh và lặp

Bước 1. Thay phần `<script>` bằng đoạn mã sau

```
<script>
  let score = 7.2;
  if (score >= 8.5) {
    console.log("Xếp loại: Giỏi");
  } else if (score >= 7) {
    console.log("Xếp loại: Khá");
  } else if (score >= 5) {
    console.log("Xếp loại: Trung bình");
  } else {
    console.log("Xếp loại: Yếu");
  }
  console.log("In các số từ 1 đến 5 bằng vòng for:");
  for (let i = 1; i <= 5; i++) {
    console.log(i);
  }
  console.log("Tính tổng từ 1 đến 5:");
  let total = 0;
  for (let i = 1; i <= 5; i++) {
    total += i;
  }
  console.log("Tổng =", total);
</script>
```

Bước 2. Chạy lại trang và xem Console

Giải thích mã

- Khỏi if...else if...else dùng để phân loại kết quả học tập.
- Vòng for thứ nhất dùng để minh họa luồng lặp.
- Vòng for thứ hai dùng để giải quyết một bài toán tính tổng đơn giản.
- Đây là bài thực hành giúp sinh viên thấy rõ vai trò của biến đếm, điều kiện dừng và cập nhật biến.

Bài thực hành 3: So sánh inline, internal, external

Cách 1. Inline

Tạo file inline.html:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Inline JavaScript</title>
</head>
<body>
  <button onclick="alert('Bạn vừa nhấn nút')">Nhấn
thử</button>
</body>
</html>
```

Cách 2. Internal

Tạo file internal.html:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Internal JavaScript</title>
</head>
<body>
  <button id="btnHello">Chào</button>
  <script>
    document.getElementById("btnHello").onclick = function
() {
      alert("Xin chào từ internal script");
    };
  </script>
</body>
</html>
```

Cách 3. External

Tạo file external.html:

```
<!DOCTYPE html>
<html lang="vi">
<head>
```

```
<meta charset="UTF-8">
<title>External JavaScript</title>
</head>
<body>
  <button id="btnExternal">Nhấn để xem lời chào</button>
  <script src="script.js"></script>
</body>
</html>
```

Tạo file script.js:

```
document.getElementById("btnExternal").onclick = function () {
  alert("Xin chào từ file script.js");
};
```

Qua ba ví dụ này, chúng ta có thể cho sinh viên nhận ra:

- Inline là cách ngắn nhất nhưng trộn lẫn HTML và JavaScript.
- Internal đã tách logic khỏi phần tử HTML, nhưng vẫn gói trong cùng một file.
- External là cách gần với phát triển dự án thực tế nhất vì mã rõ ràng, dễ mở rộng.

2.2. Làm việc với DOM

Sau khi đã nắm được biến, kiểu dữ liệu, câu lệnh điều kiện và vòng lặp trong JavaScript, chúng ta cần tiến thêm một bước rất quan trọng là điều khiển trang HTML bằng mã lệnh. Để làm được điều đó, JavaScript không thao tác trực tiếp với “mã nguồn HTML thô”, mà làm việc thông qua một mô hình biểu diễn có cấu trúc gọi là DOM.

DOM là nền tảng cho phần lớn các thao tác tương tác trên web như thay đổi tiêu đề, cập nhật nội dung văn bản, thêm sản phẩm vào danh sách, xóa một dòng dữ liệu, đổi màu nút bấm, hiện thông báo lỗi hoặc tạo ra các thành phần giao diện mới khi người dùng thao tác. Vì vậy, nếu xem JavaScript là bộ xử lý logic, thì DOM chính là cầu nối giữa logic lập trình và giao diện hiển thị.

2.2.1. Khái niệm DOM và cây cấu trúc tài liệu.

a. DOM là gì

DOM là viết tắt của **Document Object Model**. Đây là mô hình đối tượng tài liệu do trình duyệt tạo ra từ mã HTML khi trang web được tải. Thay vì coi trang HTML chỉ là các dòng văn bản, trình duyệt chuyển tài liệu đó thành một cấu trúc dạng cây gồm các đối tượng mà JavaScript có thể truy cập và điều khiển.

Nói cách khác, DOM cho phép chúng ta:

- tìm đến một phần tử HTML cụ thể,
- đọc nội dung hoặc thuộc tính của nó,
- thay đổi nội dung hoặc giao diện,
- tạo thêm phần tử mới,
- xóa phần tử cũ,
- gắn hành vi tương tác với người dùng.

Ví dụ, từ đoạn HTML:

```
<body>
  <h1>Trang học JavaScript</h1>
  <p>Nội dung mở đầu</p>
</body>
```

trình duyệt sẽ biểu diễn thành các nút trong cây DOM. Mỗi thẻ HTML như `<body>`, `<h1>`, `<p>` trở thành một đối tượng có thể thao tác bằng JavaScript.

b. Ý nghĩa của “cây cấu trúc tài liệu”

Trang HTML có cấu trúc lồng nhau. Một phần tử có thể chứa nhiều phần tử con, và chính đặc điểm này tạo nên mô hình cây. Ví dụ:

```
<!DOCTYPE html>
<html>
<head>
  <title>Ví dụ DOM</title>
</head>
<body>
  <div id="content">
    <h1>Tiêu đề chính</h1>
    <p>Đây là đoạn văn.</p>
  </div>
</body>
</html>
```

Cấu trúc cây có thể hình dung như sau:

Nút cha	Nút con
html	head, body
head	title
body	div

div	h1, p
-----	-------

Bảng 4.8: Quan hệ nút trong cây DOM

Trong mô hình này:

- html là nút gốc chính của tài liệu.
- body là phần thân hiển thị nội dung trên trang.
- div là phần tử cha chứa h1 và p.
- h1 và p là phần tử con của div.

Cách nhìn theo cây giúp chúng ta hiểu rõ quan hệ cha – con – anh em giữa các phần tử. Đây là điều rất quan trọng khi sinh viên bắt đầu thao tác với DOM phức tạp hơn.

c. Mỗi phần tử HTML là một đối tượng

Trong DOM, phần tử HTML không còn chỉ là văn bản đánh dấu mà trở thành đối tượng JavaScript có thuộc tính và phương thức. Ví dụ, một phần tử `<h1>` có thể có:

- thuộc tính như id, class, title,
- nội dung văn bản,
- các phương thức như thay đổi nội dung, thêm class, xóa phần tử.

Ví dụ:

```
<h1 id="mainTitle">Học DOM</h1>
```

JavaScript có thể truy cập và thay đổi như sau:

```
const title = document.getElementById("mainTitle");
title.textContent = "Học DOM với JavaScript";
```

Ở đây:

- document đại diện cho toàn bộ tài liệu HTML hiện tại.
- getElementById("mainTitle") tìm phần tử có id="mainTitle".
- textContent cho phép thay đổi nội dung văn bản.

d. Vai trò của đối tượng document

Khi làm việc với DOM, đối tượng trung tâm là document. Đây là điểm bắt đầu để truy cập mọi phần tử trong trang.

Một số thao tác tiêu biểu:

```
document.getElementById("...");
document.getElementsByClassName("...");
document.querySelector("...");
document.querySelectorAll("...");
```

Chúng ta có thể hiểu đơn giản rằng document giống như “cửa vào” của toàn bộ cây cấu trúc tài liệu.

2.2.2. Chọn phần tử bằng id, class, selector; thay đổi nội dung, thuộc tính, lớp CSS

Phần này là nội dung cốt lõi của thao tác DOM. Sau khi hiểu rằng trang HTML là một cây các đối tượng, chúng ta cần học cách xác định đúng phần tử muốn thao tác, sau đó thay đổi dữ liệu hoặc trạng thái hiển thị của nó.

a. Chọn phần tử bằng id

id là định danh duy nhất của một phần tử trong trang. Khi cần thao tác với một đối tượng cụ thể, id là cách chọn trực tiếp và rõ ràng nhất.

Cú pháp

```
document.getElementById("tenId")
```

Ví dụ

```
<h1 id="title">Tiêu đề ban đầu</h1>
const titleElement = document.getElementById("title");
titleElement.textContent = "Tiêu đề đã được thay đổi";
```

Giải thích

- getElementById("title") trả về phần tử có id="title".
- textContent thay đổi phần văn bản nằm trong thẻ.

b. Chọn phần tử bằng class

Khi nhiều phần tử có chung vai trò trình bày hoặc cần xử lý giống nhau, chúng ta thường dùng class.

Cú pháp

```
document.getElementsByClassName("tenClass")
```

Kết quả trả về là một tập hợp các phần tử, không phải một phần tử đơn lẻ. Vì vậy, thường cần truy cập từng phần tử bằng chỉ số hoặc lặp qua danh sách.

Ví dụ

```
<p class="note">Ghi chú 1</p>
<p class="note">Ghi chú 2</p>
<p class="note">Ghi chú 3</p>
const notes = document.getElementsByClassName("note");
notes[0].textContent = "Ghi chú đầu tiên đã được cập nhật";
```

Giải thích

- notes là danh sách các phần tử có class note.
- notes[0] là phần tử đầu tiên.

c. Chọn phần tử bằng `querySelector`

`querySelector` là cách chọn phần tử rất linh hoạt vì nó dùng cú pháp CSS selector. Đây là hướng thao tác hiện đại, thống nhất và rất phù hợp cho giảng dạy sau khi sinh viên đã học CSS cơ bản.

Cú pháp

```
document.querySelector("bo_chon_css")
```

Hàm này chỉ trả về phần tử đầu tiên khớp với bộ chọn. Ví dụ

```
<h2 class="section-title">Mục học tập</h2>
const sectionTitle = document.querySelector(".section-title");
sectionTitle.textContent = "Nội dung mục đã cập nhật";
```

d. Chọn nhiều phần tử bằng `querySelectorAll`

Khi cần chọn tất cả phần tử khớp với một selector, chúng ta dùng `querySelectorAll`. Ví dụ

```
<li class="item">HTML</li>
<li class="item">CSS</li>
<li class="item">JavaScript</li>
const items = document.querySelectorAll(".item");
items.forEach(function(item) {
    item.textContent = item.textContent + " - đã học";
});
```

Giải thích

- `querySelectorAll(".item")` lấy toàn bộ phần tử có class `item`.
- `forEach` duyệt từng phần tử trong danh sách.
- Mỗi phần tử được nối thêm chuỗi `" - đã học"`.

Bảng so sánh các cách chọn phần tử

Cách chọn	Cú pháp	Phạm vi sử dụng	Đặc điểm
Theo id	<code>getElementById("id")</code>	Một phần tử duy nhất	Nhanh, rõ ràng, phổ biến
Theo class	<code>getElementsByClassName("class")</code>	Nhiều phần tử	Trả về tập hợp dạng danh sách
Theo selector đầu tiên	<code>querySelector("selector")</code>	Phần tử đầu tiên khớp	Linh hoạt, dùng cú pháp CSS

Theo tất cả selector	<code>querySelectorAll("selector")</code>	Tất cả phần tử khớp	Phù hợp khi cần lặp và xử lý hàng loạt
----------------------	---	---------------------	--

Bảng 4.9: So sánh các cách chọn phần tử DOM

f. Thay đổi nội dung phần tử

Có nhiều cách thay đổi nội dung, nhưng trong phần nhập môn chúng ta nên tập trung vào `textContent` và giới thiệu thêm `innerHTML`.

Dùng `textContent`

```
const title = document.getElementById("title");
title.textContent = "Nội dung mới";
```

`textContent` chỉ thay đổi văn bản thuần, an toàn và dễ hiểu.

Dùng `innerHTML`

```
const box = document.getElementById("box");
box.innerHTML = "<strong>Nội dung in đậm</strong>";
```

`innerHTML` cho phép chèn cả mã HTML vào bên trong phần tử. Tuy nhiên, trong dạy học cơ bản cần nhấn mạnh rằng cách này mạnh nhưng phải dùng cẩn thận để tránh lỗi cấu trúc và nguy cơ chèn nội dung không an toàn nếu dữ liệu đến từ người dùng.

g. Thay đổi thuộc tính của phần tử

Phần tử HTML có thể có nhiều thuộc tính như `src`, `href`, `alt`, `title`, `value`, `placeholder`. JavaScript có thể đọc hoặc thay đổi các thuộc tính này. Ví dụ thay đổi ảnh

```

const image = document.getElementById("mainImage");
image.src = "new.jpg";
image.alt = "Ảnh mới";
```

Ví dụ thay đổi liên kết

```
<a id="myLink" href="#">Trang chủ</a>
const link = document.getElementById("myLink");
link.href = "https://example.com";
link.textContent = "Đi tới website";
```

h. Thay đổi class CSS

Một trong những thao tác DOM phổ biến nhất là thêm, xóa hoặc bật tắt class CSS để đổi giao diện mà không cần sửa trực tiếp thuộc tính `style`. Ví dụ HTML và CSS

```
<p id="message">Thông báo hệ thống</p>
.highlight {
  color: red;
  font-weight: bold;
}
```

Thêm class

```
const message = document.getElementById("message");
message.classList.add("highlight");
```

Xóa class

```
message.classList.remove("highlight");
```

Bật/tắt class

```
message.classList.toggle("highlight");
```

i. Ý nghĩa kỹ thuật của classList

classList là công cụ chuẩn để thao tác với lớp CSS. So với việc gán trực tiếp chuỗi className, classList rõ ràng hơn và ít gây ghi đè nhầm các class khác.

Ví dụ:

```
message.className = "highlight";
```

có thể làm mất các class cũ. Trong khi đó:

```
message.classList.add("highlight");
```

chỉ thêm class mới mà không phá vỡ class hiện có.

2.2.3. Tạo, chèn, xóa phần tử trên trang bằng JavaScript.

Khi trang web cần thay đổi giao diện theo dữ liệu hoặc hành vi người dùng, chúng ta không thể chỉ sửa các phần tử đã có sẵn. Lúc đó, JavaScript cần có khả năng tạo mới phần tử, chèn vào vị trí phù hợp và xóa khi không còn cần thiết.

a. Tạo phần tử mới

Chúng ta dùng document.createElement() để tạo một node HTML mới. Ví dụ

```
const newItem = document.createElement("li");
newItem.textContent = "JavaScript DOM";
```

Ở đây:

- createElement("li") tạo ra một thẻ mới nhưng chưa xuất hiện trên trang.
- textContent gán nội dung văn bản cho phần tử này.

b. Chèn phần tử vào trang

Sau khi tạo phần tử, cần đưa nó vào cây DOM bằng các phương thức như `appendChild()` hoặc `append()`. Ví dụ HTML

```
<ul id="courseList">
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

Ví dụ JavaScript

```
const list = document.getElementById("courseList");
const newItem = document.createElement("li");
newItem.textContent = "JavaScript";
list.appendChild(newItem);
```

Giải thích

- `list` là phần tử `<ul>`.
- `newItem` là phần tử `<li>` mới.
- `appendChild(newItem)` chèn `newItem` vào cuối danh sách.

Sau khi chạy, danh sách sẽ có thêm mục "JavaScript".

c. Chèn phần tử vào vị trí cụ thể

Ngoài việc thêm vào cuối, chúng ta có thể chèn vào trước một phần tử khác bằng `insertBefore()`. Ví dụ

```
const list = document.getElementById("courseList");
const firstItem = list.children[0];
const newItem = document.createElement("li");
newItem.textContent = "Mở đầu khóa học";
list.insertBefore(newItem, firstItem);
```

Kết quả là mục mới được chèn lên đầu danh sách.

d. Xóa phần tử khỏi trang

Khi cần loại bỏ một phần tử, chúng ta có thể dùng `remove()`. Ví dụ

```
<p id="oldMessage">Thông báo cũ</p>
const oldMessage = document.getElementById("oldMessage");
oldMessage.remove();
```

Sau khi thực thi, phần tử `<p>` sẽ biến mất khỏi trang.

e. Xóa phần tử con từ phần tử cha

Một số tình huống yêu cầu xóa node con thông qua phần tử cha.

```
const list = document.getElementById("courseList");
const firstItem = list.children[0];
list.removeChild(firstItem);
```

Bảng tóm tắt các thao tác DOM tạo/chèn/xóa

Thao tác	Phương thức	Mục đích
Tạo phần tử	document.createElement("tag")	Tạo node mới
Thêm vào cuối	appendChild(node)	Chèn node vào cuối phần tử cha
Chèn trước node khác	insertBefore(newNode, refNode)	Chèn vào vị trí xác định
Xóa trực tiếp	node.remove()	Loại bỏ node khỏi DOM
Xóa node con	parent.removeChild(child)	Xóa node con từ node cha

Bảng 4.10: Các thao tác DOM tạo, chèn, xóa phần tử

Kế tiếp, chúng ta sẽ thực hiện một số bài thực hành để làm quen các khái niệm đã tìm hiểu trên.

Bài thực hành 1: Quan sát DOM và thay đổi nội dung

Bước 1. Tạo file index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Thực hành DOM cơ bản</title>
  <style>
    .highlight {
      color: blue;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1 id="title">Tiêu đề ban đầu</h1>
```

```

<p class="desc">Đây là đoạn mô tả thứ nhất.</p>
<p class="desc">Đây là đoạn mô tả thứ hai.</p>
<button id="changeBtn">Thay đổi nội dung</button>
<script>
    const title = document.getElementById("title");
    const descriptions =
document.querySelectorAll(".desc");
    const changeBtn = document.getElementById("changeBtn");
    changeBtn.onclick = function () {
        title.textContent = "Tiêu đề đã được cập nhật bằng
JavaScript";
        title.classList.add("highlight");
        descriptions.forEach(function (item, index) {
            item.textContent = "Đoạn mô tả số " + (index +
1) + " đã được cập nhật.";
        });
    };
</script>
</body>
</html>

```

Bước 2. Chạy bằng Live Server hoặc mở trực tiếp trên trình duyệt

Bước 3. Nhấn nút “Thay đổi nội dung”

Kết quả cần quan sát

- Tiêu đề h1 đổi nội dung.
- Tiêu đề được thêm class highlight.
- Hai đoạn văn bản có class desc được cập nhật lại.

Bài thực hành 2: Thay đổi thuộc tính

Mã HTML

```

<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <title>Thay đổi thuộc tính</title>
</head>
<body>
    
  <br><br>
  <a id="link" href="#">Liên kết ban đầu</a>
  <br><br>
  <button id="updateBtn">Cập nhật thuộc tính</button>
  <script>
    const photo = document.getElementById("photo");
    const link = document.getElementById("link");
    const updateBtn = document.getElementById("updateBtn");
    updateBtn.onclick = function () {
      photo.src = "https://via.placeholder.com/200";
      photo.alt = "Ảnh đã cập nhật";

      link.href = "https://developer.mozilla.org";
      link.textContent = "Mở tài liệu MDN";
      link.target = "_blank";
    };
  </script>
</body>
</html>

```

Giải thích

- Nút bấm điều khiển việc cập nhật src và alt của ảnh.
- Đồng thời cập nhật href, textContent và target của liên kết.

Bài thực hành 3: Tạo, chèn và xóa phần tử

Mã HTML

```

<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Quản lý danh sách công việc</title>
  <style>
    body {
      font-family: Arial, sans-serif;
    }
    li {
      margin: 8px 0;
    }
  </style>

```

```
    }
    button {
        margin-left: 8px;
    }
</style>
</head>
<body>
    <h1>Danh sách công việc</h1>
    <input type="text" id="taskInput" placeholder="Nhập công
việc">
    <button id="addBtn">Thêm công việc</button>
    <ul id="taskList"></ul>
    <script>
        const taskInput = document.getElementById("taskInput");
        const addBtn = document.getElementById("addBtn");
        const taskList = document.getElementById("taskList");
        addBtn.onclick = function () {
            const taskText = taskInput.value.trim();
            if (taskText === "") {
                alert("Vui lòng nhập nội dung");
                return;
            }
            const li = document.createElement("li");
            li.textContent = taskText;
            const deleteBtn = document.createElement("button");
            deleteBtn.textContent = "Xóa";
            deleteBtn.onclick = function () {
                li.remove();
            };
            li.appendChild(deleteBtn);
            taskList.appendChild(li);
            taskInput.value = "";
            taskInput.focus();
        };
    </script>
```

```
</body>
</html>
```

Quy trình thực hành

Bước 1. Nhập nội dung công việc vào ô văn bản.

Bước 2. Nhấn nút “Thêm công việc”.

Bước 3. Quan sát một phần tử li mới xuất hiện trong danh sách.

Bước 4. Nhấn nút “Xóa” tương ứng để loại bỏ công việc.

Bài này tích hợp nhiều thao tác DOM trong một tình huống sát thực tế:

- lấy dữ liệu từ ô nhập,
- kiểm tra hợp lệ đầu vào,
- tạo phần tử mới,
- chèn phần tử vào cây DOM,
- tạo nút chức năng động,
- xóa phần tử khi có yêu cầu.

Lỗi thường gặp khi học DOM

Lỗi	Nguyên nhân	Cách khắc phục
Cannot read properties of null	Chọn sai id hoặc script chạy trước khi phần tử được tải	Kiểm tra tên id, đặt script cuối body
Không thay đổi được nội dung	Chọn sai phần tử hoặc viết sai thuộc tính	Kiểm tra lại biến tham chiếu và thuộc tính như <code>textContent</code>
Không thêm được phần tử mới	Quên <code>appendChild()</code> hoặc phần tử cha không đúng	Xác định đúng node cha cần chèn
Nút xóa không hoạt động	Chưa gán <code>onclick</code> cho nút mới tạo	Gắn sự kiện ngay sau khi tạo nút
Giao diện không đổi khi thêm class	Chưa định nghĩa CSS tương ứng	Kiểm tra class trong CSS và JavaScript có khớp không

Bảng 4.11: Lỗi thường gặp khi học DOM

2.3. Xử lý sự kiện người dùng

Sau khi đã nắm được DOM và các thao tác chọn, sửa, thêm hoặc xóa phần tử, chúng ta cần đi tiếp đến một nội dung có tính thực hành rất cao là xử lý sự

kiện người dùng. Một trang web chỉ thực sự trở nên tương tác khi nó biết phản ứng trước hành động của người dùng như nhấn nút, nhập dữ liệu, thay đổi lựa chọn, gửi biểu mẫu hay rê chuột. Toàn bộ các phản ứng đó đều được xây dựng dựa trên cơ chế sự kiện.

Trong lập trình giao diện web, sự kiện là cầu nối giữa hành vi của người dùng và mã lệnh JavaScript. Khi một sự kiện xảy ra, JavaScript có thể thực hiện một hàm xử lý tương ứng để cập nhật giao diện, kiểm tra dữ liệu, hiển thị thông báo hoặc thay đổi trạng thái của thành phần giao diện. Vì vậy, nội dung này có ý nghĩa đặc biệt quan trọng trong việc hình thành tư duy lập trình hướng tương tác.

2.3.1. Sự kiện *click*, *change*, *keyup*, *submit*,... và cách gắn sự kiện cho phần tử.

a. Khái niệm sự kiện trong JavaScript

Sự kiện là một hành động hoặc biến cố xảy ra trên trình duyệt mà JavaScript có thể lắng nghe và xử lý. Sự kiện có thể phát sinh từ người dùng, từ trình duyệt hoặc từ một phần tử HTML. Một số ví dụ quen thuộc:

- người dùng nhấn nút,
- nhập dữ liệu vào ô văn bản,
- thay đổi lựa chọn trong danh sách,
- gửi biểu mẫu,
- di chuyển chuột,
- tải xong trang web.

Trong phạm vi mục này, chúng ta tập trung vào các sự kiện cơ bản, xuất hiện thường xuyên trong giáo trình nhập môn.

b. Một số sự kiện thông dụng

Sự kiện	Khi nào xảy ra	Phần tử thường dùng
click	Khi người dùng nhấn chuột vào phần tử	button, a, div, img
change	Khi giá trị phần tử thay đổi và được xác nhận	input, select, textarea
keyup	Khi người dùng nhả phím trên bàn phím	input, textarea, document
submit	Khi biểu mẫu được gửi	form
focus	Khi phần tử nhận con trỏ nhập liệu	input, textarea

blur	Khi phần tử mất con trỏ nhập liệu	input, textarea
mouseover	Khi chuột đi vào phần tử	Hầu hết các phần tử
mouseout	Khi chuột rời khỏi phần tử	Hầu hết các phần tử

Bảng 4.12: Một số sự kiện thông dụng

Sự kiện click

click là sự kiện cơ bản nhất, thường được dùng để xử lý hành động nhấn nút. Ví dụ

```
<button id="btnHello">Nhấn vào đây</button>
const btnHello = document.getElementById("btnHello");
btnHello.onclick = function () {
    alert("Bạn vừa nhấn nút");
};
```

Phân tích

- Người dùng nhấn nút.
- Sự kiện click xảy ra.
- Hàm xử lý được gọi.
- Trình duyệt hiển thị hộp thông báo.

Sự kiện change

change thường dùng với ô nhập liệu, hộp chọn hoặc danh sách lựa chọn. Sự kiện này xảy ra khi giá trị của phần tử thay đổi và việc thay đổi đó đã được xác nhận. Ví dụ

```
<select id="colorSelect">
    <option value="">-- Chọn màu --</option>
    <option value="red">Đỏ</option>
    <option value="blue">Xanh</option>
</select>
<p id="result"></p>
const colorSelect = document.getElementById("colorSelect");
const result = document.getElementById("result");
colorSelect.onchange = function () {
    result.textContent = "Màu đã chọn: " + colorSelect.value;
};
```

Sự kiện keyup

keyup xảy ra khi người dùng nhả một phím trên bàn phím. Đây là sự kiện rất hữu ích khi cần phản hồi ngay trong quá trình nhập, chẳng hạn hiển thị nội dung đang gõ, đếm ký tự hoặc kiểm tra dữ liệu thời gian thực. Ví dụ

```
<input type="text" id="nameInput" placeholder="Nhập tên">
<p id="preview"></p>
const nameInput = document.getElementById("nameInput");
const preview = document.getElementById("preview");
nameInput.onkeyup = function () {
    preview.textContent = "Bạn đang nhập: " + nameInput.value;
};
```

Sự kiện submit

submit được dùng với phần tử <form>. Khi người dùng gửi biểu mẫu, sự kiện này xảy ra. Ví dụ

```
<form id="loginForm">
    <input type="text" id="username" placeholder="Tên đăng
nhập">
    <button type="submit">Gửi</button>
</form>
const loginForm = document.getElementById("loginForm");
loginForm.onsubmit = function (event) {
    event.preventDefault();
    alert("Biểu mẫu đã được xử lý bằng JavaScript");
};
```

Phân tích

- Mặc định, khi form được gửi, trình duyệt thường tải lại trang hoặc chuyển sang URL khác.
- event.preventDefault() ngăn hành vi mặc định đó.
- Đây là thao tác rất quan trọng trong các biểu mẫu web hiện đại.

c. Cách gắn sự kiện cho phần tử

Có nhiều cách gắn sự kiện, nhưng trong học phần cơ bản chúng ta nên trình bày từ đơn giản đến chuẩn thực tế.

Gắn trực tiếp trong HTML

Đây là cách viết sự kiện ngay trong thuộc tính của phần tử HTML.

```
<button onclick="alert('Xin chào')">Nhấn nút</button>
```

Đặc điểm

- Dễ minh họa.
- Ngắn gọn với ví dụ rất nhỏ.
- Không phù hợp khi dự án lớn vì trộn lẫn HTML và JavaScript.

Gắn qua thuộc tính sự kiện trong JavaScript

```
<button id="btnTest">Nhấn thử</button>
const btnTest = document.getElementById("btnTest");
btnTest.onclick = function () {
    console.log("Đã nhấn nút");
};
```

Đặc điểm

- Tách HTML và JavaScript rõ hơn.
- Dễ học với người mới bắt đầu.
- Mỗi thuộc tính chỉ chứa được một hàm xử lý chính tại một thời điểm.

Gắn bằng addEventListener

Đây là cách chuẩn và linh hoạt hơn, thường được khuyến nghị trong thực hành hiện đại.

```
<button id="btnEvent">Sự kiện chuẩn</button>
const btnEvent = document.getElementById("btnEvent");
btnEvent.addEventListener("click", function () {
    console.log("Sự kiện được gắn bằng addEventListener");
});
```

Ưu điểm

- Có thể gắn nhiều hàm xử lý cho cùng một sự kiện.
- Tách biệt tốt giữa cấu trúc HTML và logic JavaScript.
- Phù hợp cho dự án thực tế hơn.

Bảng so sánh các cách gắn sự kiện

Cách gắn	Ví dụ	Ưu điểm	Hạn chế	Khuyến nghị
Inline trong HTML	onclick="..."	Nhanh, dễ minh họa	Trộn HTML và JS	Chỉ dùng cho ví dụ rất ngắn
Thuộc tính JS	element.onclick = ...	Dễ hiểu, dễ học	Kém linh hoạt hơn	Phù hợp giai đoạn nhập môn

addEventListener	element.addEventListener(...)	Chuẩn, mở rộng tốt	Cú pháp dài hơn chút	Nên dùng trong thực hành chuẩn
------------------	-------------------------------	--------------------	----------------------	--------------------------------

Bảng 4.13: So sánh các cách gắn sự kiện

2.3.2. Ví dụ: nút bấm thay đổi nội dung, ẩn/hiện khối nội dung, hiển thị thông báo

Phần này giúp chúng ta đưa khái niệm sự kiện vào các tình huống gần với thực tế giao diện web. Thay vì chỉ quan sát mã ngắ, sinh viên sẽ thấy sự kiện làm thay đổi trạng thái giao diện như thế nào.

Ví dụ 1: Nút bấm thay đổi nội dung

HTML

```
<h2 id="title">Nội dung ban đầu</h2>
<button id="changeBtn">Thay đổi nội dung</button>
```

JavaScript

```
const title = document.getElementById("title");
const changeBtn = document.getElementById("changeBtn");
changeBtn.addEventListener("click", function () {
    title.textContent = "Nội dung đã được thay đổi";
});
```

Giải thích

- Khi nhấn nút, sự kiện click được kích hoạt.
- Hàm xử lý sẽ thay đổi văn bản của tiêu đề.
- Đây là ví dụ cơ bản nhất về kết hợp DOM và sự kiện.

Ví dụ 2: Ẩn hoặc hiện khối nội dung

HTML

```
<button id="toggleBtn">Ẩn/Hiện nội dung</button>
<div id="contentBox">
    Đây là khối nội dung cần điều khiển hiển thị.
</div>
```

CSS

```
#contentBox {
    margin-top: 10px;
    padding: 10px;
    border: 1px solid #ccc;
}
```

```
.hidden {  
    display: none;  
}
```

JavaScript

```
const toggleBtn = document.getElementById("toggleBtn");  
const contentBox = document.getElementById("contentBox");  
toggleBtn.addEventListener("click", function () {  
    contentBox.classList.toggle("hidden");  
});
```

Phân tích kỹ thuật

- `classList.toggle("hidden")` sẽ thêm class `hidden` nếu chưa có, hoặc xóa class đó nếu đã có.
- Vì `.hidden { display: none; }`, phần tử sẽ được ẩn hoặc hiện luân phiên.
- Cách làm này tốt hơn việc gán trực tiếp `style.display` nhiều lần vì tách được logic và trình bày.

Ví dụ 3: Hiện thị thông báo khi nhập dữ liệu

HTML

```
<input type="text" id="emailInput" placeholder="Nhập email">  
<p id="message"></p>
```

JavaScript

```
const emailInput = document.getElementById("emailInput");  
const message = document.getElementById("message");  
emailInput.addEventListener("keyup", function () {  
    if (emailInput.value.trim() === "") {  
        message.textContent = "Vui lòng nhập email";  
    } else {  
        message.textContent = "Dữ liệu đang được nhập";  
    }  
});
```

Ví dụ này cho thấy giao diện có thể phản hồi ngay trong lúc người dùng thao tác. Đây là cơ sở để sau này phát triển các chức năng kiểm tra biểu mẫu, gợi ý tìm kiếm hoặc lọc dữ liệu tức thời.

Ví dụ 4: Xử lý submit trên biểu mẫu

HTML

```
<form id="contactForm">  
    <input type="text" id="fullName" placeholder="Họ tên">
```

```
<button type="submit">Gửi thông tin</button>
</form>
<p id="submitResult"></p>
```

JavaScript

```
const contactForm = document.getElementById("contactForm");
const fullName = document.getElementById("fullName");
const submitResult = document.getElementById("submitResult");
contactForm.addEventListener("submit", function (event) {
    event.preventDefault();
    if (fullName.value.trim() === "") {
        submitResult.textContent = "Bạn chưa nhập họ tên.";
        return;
    }
    submitResult.textContent = "Đã tiếp nhận thông tin của: " +
    fullName.value;
});
```

Phân tích

- submit không chỉ là nhấn nút mà là gửi toàn bộ biểu mẫu.
- preventDefault() giúp chúng ta kiểm soát luồng xử lý trên JavaScript mà không làm tải lại trang.
- Đây là mô hình làm việc phổ biến của các form hiện đại.

2.3.3. Xây dựng các thành phần tương tác đơn giản (tab, accordion, menu thu gọn)

Khi đã hiểu sự kiện và DOM, chúng ta có thể xây dựng các thành phần giao diện có tính tương tác. Phần này đặc biệt hữu ích vì gắn lý thuyết với các thành phần xuất hiện thường xuyên trên website thực tế.

a. Thành phần tab

Tab cho phép chia nội dung thành nhiều nhóm, chỉ hiển thị một nhóm tại một thời điểm. Đây là cấu trúc phổ biến trong giao diện quản trị, tài liệu học tập hoặc trang giới thiệu nhiều nội dung.

HTML

```
<div class="tabs">
    <button class="tab-btn active" data-tab="tab1">Giới
    thiệu</button>
    <button class="tab-btn" data-tab="tab2">Nội dung</button>
```

```
    <button class="tab-btn" data-tab="tab3">Liên hệ</button>
</div>

<div id="tab1" class="tab-content active-content">Đây là phần
giới thiệu.</div>
<div id="tab2" class="tab-content">Đây là phần nội dung bài
học.</div>
<div id="tab3" class="tab-content">Đây là phần liên hệ.</div>
```

CSS

```
.tab-content {
    display: none;
    margin-top: 10px;
    padding: 10px;
    border: 1px solid #ccc;
}
.active-content {
    display: block;
}
.active {
    font-weight: bold;
}
```

JavaScript

```
const tabButtons = document.querySelectorAll(".tab-btn");
const tabContents = document.querySelectorAll(".tab-content");
tabButtons.forEach(function (button) {
    button.addEventListener("click", function () {
        const targetId = button.dataset.tab;
        tabButtons.forEach(function (btn) {
            btn.classList.remove("active");
        });
        tabContents.forEach(function (content) {
            content.classList.remove("active-content");
        });
        button.classList.add("active");
        document.getElementById(targetId).classList.add("active-
content");
    });
});
```

```
});  
});
```

Giải thích

- Mỗi nút tab có thuộc tính data-tab chứa id của vùng nội dung cần hiển thị.
- Khi người dùng nhấn nút:
 - xóa trạng thái active của tất cả nút,
 - ẩn tất cả vùng nội dung,
 - kích hoạt nút hiện tại,
 - hiển thị vùng nội dung tương ứng.
- Đây là ví dụ rất điển hình của việc phối hợp giữa selector, class CSS và sự kiện click.

b. Thành phần accordion

Accordion là thành phần cho phép mở hoặc đóng từng khối nội dung. Nó thường được dùng trong câu hỏi thường gặp, danh sách bài học, phần mô tả chi tiết hoặc menu nhiều cấp.

HTML

```
<div class="accordion-item">  
  <button class="accordion-btn">Mục 1</button>  
  <div class="accordion-content">Nội dung của mục 1</div>  
</div>  
<div class="accordion-item">  
  <button class="accordion-btn">Mục 2</button>  
  <div class="accordion-content">Nội dung của mục 2</div>  
</div>
```

CSS

```
.accordion-content {  
  display: none;  
  padding: 10px;  
  border: 1px solid #ccc;  
  margin-bottom: 8px;  
}  
.accordion-content.show {  
  display: block;  
}
```

JavaScript

```
const accordionButtons = document.querySelectorAll(".accordion-
btn");
accordionButtons.forEach(function (button) {
    button.addEventListener("click", function () {
        const content = button.nextElementSibling;
        content.classList.toggle("show");
    });
});
```

Giải thích

- `nextElementSibling` lấy phần tử đứng ngay sau nút, tức khối nội dung liên quan.
- `toggle("show")` giúp mở hoặc đóng vùng nội dung.
- Cấu trúc này ngắn gọn nhưng rất hiệu quả để dạy sinh viên về quan hệ giữa phần tử đang thao tác và phần tử kế cận trong DOM.

c. Menu thu gọn

Menu thu gọn thường xuất hiện trên giao diện di động hoặc các thanh điều hướng nhỏ gọn. Khi nhấn nút menu, danh sách liên kết sẽ hiện ra hoặc thu lại.

HTML

```
<button id="menuToggle">☰ Menu</button>
<nav id="mobileMenu" class="menu">
    <a href="#">Trang chủ</a>
    <a href="#">Khóa học</a>
    <a href="#">Liên hệ</a>
</nav>
```

CSS

```
.menu {
    display: none;
    margin-top: 10px;
}
.menu.show {
    display: block;
}
.menu a {
    display: block;
    margin: 6px 0;
}
```

JavaScript

```
const menuToggle = document.getElementById("menuToggle");
const mobileMenu = document.getElementById("mobileMenu");
menuToggle.addEventListener("click", function () {
    mobileMenu.classList.toggle("show");
});
```

Ví dụ này giúp người học hiểu rằng nhiều thành phần giao diện hiện đại thực chất được xây dựng từ:

- một trạng thái hiển thị,
- một class CSS,
- một sự kiện click,
- một thao tác bật hoặc tắt trạng thái đó.

Bảng tổng hợp các thành phần tương tác đơn giản

Thành phần	Sự kiện chính	Kỹ thuật DOM chính	Mục đích
Nút đổi nội dung	click	textContent	Cập nhật văn bản
Khối ẩn/hiện	click	classList.toggle()	Điều khiển hiển thị
Thông báo nhập liệu	keyup, change	Đọc value, cập nhật nội dung	Phản hồi tức thời
Form gửi dữ liệu	submit	preventDefault()	Kiểm soát gửi biểu mẫu
Tab	click	Selector, class active	Chuyển vùng nội dung
Accordion	click	nextElementSibling, toggle class	Mở đóng từng khối
Menu thu gọn	click	toggle class	Hiện hoặc ẩn menu

Bảng 4.14: Tổng hợp các thành phần tương tác đơn giản

Kế tiếp, chúng ta sẽ thực hiện một số bài thực hành để làm quen các khái niệm đã tìm hiểu trên.

Bài thực hành 1: Nút thay đổi nội dung và hiển thị thông báo

Bước 1. Tạo file index.html

```
<!DOCTYPE html>
<html lang="vi">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Thực hành sự kiện cơ bản</title>
</head>
<body>
  <h1 id="title">Chào mừng đến với JavaScript</h1>
  <button id="changeBtn">Đổi tiêu đề</button>
  <br><br>
  <input type="text" id="nameInput" placeholder="Nhập tên của
bạn">
  <p id="message"></p>
  <script>
    const title = document.getElementById("title");
    const changeBtn = document.getElementById("changeBtn");
    const nameInput = document.getElementById("nameInput");
    const message = document.getElementById("message");
    changeBtn.addEventListener("click", function () {
      title.textContent = "Tiêu đề đã được cập nhật";
    });
    nameInput.addEventListener("keyup", function () {
      if (nameInput.value.trim() === "") {
        message.textContent = "Bạn chưa nhập tên.";
      } else {
        message.textContent = "Xin chào, " +
nameInput.value;
      }
    });
  </script>
</body>
</html>

```

Bước 2. Chạy trên trình duyệt

Mở file bằng Live Server hoặc mở trực tiếp trong trình duyệt.

Bước 3. Thực hành thao tác

- Nhấn nút để đổi tiêu đề.

- Nhập tên vào ô nhập để quan sát phản hồi tức thời.
Kết quả mong đợi
- Sự kiện click phù hợp với nút bấm.
- Sự kiện keyup phù hợp với ô nhập liệu thời gian thực.
- DOM và sự kiện luôn đi cùng nhau trong giao diện tương tác.

Bài thực hành 2: Xây dựng accordion đơn giản

Tạo file index.html có nội dung:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Accordion đơn giản</title>
  <style>
    .accordion-content {
      display: none;
      padding: 10px;
      border: 1px solid #ccc;
      margin-bottom: 10px;
    }
    .accordion-content.show {
      display: block;
    }
  </style>
</head>
<body>
  <button class="accordion-btn">Bài học 1</button>
  <div class="accordion-content">Nội dung bài học 1</div>
  <button class="accordion-btn">Bài học 2</button>
  <div class="accordion-content">Nội dung bài học 2</div>
  <script>
    const buttons = document.querySelectorAll(".accordion-
btn");
    buttons.forEach(function (button) {
      button.addEventListener("click", function () {
        const content = button.nextElementSibling;
        content.classList.toggle("show");
      });
    });
  </script>
</body>
</html>
```

```
        });  
    });  
</script>  
</body>  
</html>
```

Điều cần chú ý

- Cấu trúc HTML phải được sắp xếp sao cho phần tử nội dung đứng ngay sau nút.
- `nextElementSibling` sẽ hoạt động đúng khi cấu trúc DOM rõ ràng.
- Đây là ví dụ rất tốt để dạy mối liên hệ giữa cấu trúc HTML và logic JavaScript.

Bài thực hành 3: Menu thu gọn

Tạo file `index.html` có nội dung:

```
<!DOCTYPE html>  
<html lang="vi">  
<head>  
    <meta charset="UTF-8">  
    <title>Menu thu gọn</title>  
    <style>  
        .menu {  
            display: none;  
            margin-top: 10px;  
        }  
        .menu.show {  
            display: block;  
        }  
        .menu a {  
            display: block;  
            margin: 6px 0;  
        }  
    </style>  
</head>  
<body>  
    <button id="menuToggle">≡ Menu</button>  
    <nav id="mobileMenu" class="menu">  
        <a href="#">Trang chủ</a>
```

```

        <a href="#">Tin tức</a>
        <a href="#">Liên hệ</a>
    </nav>
    <script>
        const menuToggle =
document.getElementById("menuToggle");
        const mobileMenu =
document.getElementById("mobileMenu");
        menuToggle.addEventListener("click", function () {
            mobileMenu.classList.toggle("show");
        });
    </script>
</body>
</html>

```

Bảng sau tổng hợp các lỗi thường gặp khi học xử lý sự kiện

Lỗi	Nguyên nhân	Cách khắc phục
Nhấn nút nhưng không có phản ứng	Chọn sai id hoặc script chưa chạy đúng thời điểm	Kiểm tra tên phần tử, đặt script cuối body
Form gửi xong bị tải lại trang	Quên event.preventDefault()	Thêm ngăn hành vi mặc định trong submit
keyup không hoạt động đúng	Gắn sai phần tử hoặc đọc sai giá trị	Gắn sự kiện trên input và kiểm tra value
toggle không làm thay đổi giao diện	Chưa có CSS cho class tương ứng	Khai báo rõ class trong CSS
Accordion hoặc tab hoạt động sai	Cấu trúc HTML không khớp logic JS	Kiểm tra quan hệ DOM giữa nút và nội dung

Bảng 4.15: Lỗi thường gặp khi học xử lý sự kiện

2.4. Kiểm tra dữ liệu nhập trên form

Trong các ứng dụng web, biểu mẫu là nơi tiếp nhận dữ liệu từ người dùng như họ tên, email, số điện thoại, mật khẩu, nội dung liên hệ hoặc thông tin đăng ký. Tuy nhiên, dữ liệu do người dùng nhập vào không phải lúc nào cũng đầy đủ và đúng định dạng. Vì vậy, trước khi xử lý hoặc gửi dữ liệu lên máy chủ, chúng ta cần thực hiện bước kiểm tra dữ liệu đầu vào.

Kiểm tra dữ liệu nhập trên form có ba mục tiêu kỹ thuật quan trọng. Thứ nhất, bảo đảm dữ liệu thu được đúng cấu trúc mong muốn. Thứ hai, hỗ trợ người dùng phát hiện lỗi sớm ngay trên giao diện. Thứ ba, hạn chế tình trạng gửi form sai, gây phát sinh lỗi xử lý ở bước sau. Trong mô đun này, chúng ta tập trung vào kiểm tra phía trình duyệt bằng JavaScript để sinh viên hiểu rõ cơ chế tương tác giữa form, DOM và sự kiện.

Cần lưu ý rằng kiểm tra bằng JavaScript phía client có ý nghĩa rất lớn về mặt trải nghiệm người dùng, nhưng trong hệ thống thực tế vẫn cần kiểm tra thêm ở phía server để bảo đảm an toàn và toàn vẹn dữ liệu. Trong phạm vi bài học này, trọng tâm của chúng ta là xây dựng quy trình kiểm tra cơ bản, rõ ràng, dễ thực hành và dễ mở rộng.

2.4.1. Kiểm tra trường bắt buộc, định dạng email đơn giản.

a. Khái niệm trường bắt buộc

Trường bắt buộc là trường mà người dùng phải nhập dữ liệu trước khi gửi form. Nếu để trống, form không được xem là hợp lệ. Ví dụ điển hình là họ tên, email, tên đăng nhập hoặc mật khẩu.

Việc kiểm tra trường bắt buộc thường là bước đầu tiên trong quy trình xác thực dữ liệu nhập. Nếu bỏ qua bước này, chương trình có thể nhận dữ liệu rỗng và gây lỗi trong các bước xử lý sau.

b. Kiểm tra dữ liệu rỗng

Trong JavaScript, khi lấy dữ liệu từ ô nhập bằng thuộc tính `value`, chúng ta thường dùng thêm `trim()` để loại bỏ khoảng trắng đầu và cuối chuỗi. Điều này giúp tránh tình huống người dùng chỉ nhập dấu cách nhưng chương trình vẫn tưởng là có dữ liệu.

Ví dụ

```
<input type="text" id="fullName" placeholder="Nhập họ tên">
<p id="nameError"></p>
const fullName = document.getElementById("fullName");
const nameError = document.getElementById("nameError");
if (fullName.value.trim() === "") {
    nameError.textContent = "Họ tên là trường bắt buộc.";
}
```

c. Kiểm tra định dạng email đơn giản

Email là dữ liệu xuất hiện rất thường xuyên trong form đăng ký, đăng nhập, liên hệ hoặc nhận thông báo. Ở mức nhập môn, chúng ta chỉ cần kiểm tra định dạng cơ bản thay vì xây dựng biểu thức quá phức tạp.

Một email hợp lệ ở mức đơn giản thường có:

- phần tên trước ký tự @,
- ký tự @,
- phần miền sau @,
- dấu chấm trong phần miền.

Ví dụ hợp lệ:

- abc@gmail.com
- sinhvien01@ctec.edu.vn

Ví dụ không hợp lệ:

- abcgmail.com
- abc@
- @gmail.com

d. Dùng biểu thức chính quy đơn giản để kiểm tra email

Một cách kiểm tra phổ biến là dùng Regular Expression.

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Phân tích mẫu kiểm tra

- ^ bắt đầu chuỗi.
- [^\s@]+ yêu cầu có ít nhất một ký tự không phải khoảng trắng và không phải @.
- @ bắt buộc có ký tự @.
- [^\s@]+ yêu cầu có phần miền phía sau.
- \. yêu cầu có dấu chấm.
- [^\s@]+ yêu cầu có phần đuôi miền.
- \$ kết thúc chuỗi.

Đây là mẫu kiểm tra đơn giản, phù hợp để dạy học phần cơ bản vì dễ hiểu và đủ dùng cho phần lớn ví dụ nhập môn.

Ví dụ kiểm tra họ tên và email

```
<form id="registerForm">
  <input type="text" id="fullName" placeholder="Nhập họ tên">
  <p id="nameError"></p>
  <input type="text" id="email" placeholder="Nhập email">
  <p id="emailError"></p>
```

```

    <button type="submit">Gửi đăng ký</button>
</form>
const registerForm = document.getElementById("registerForm");
const fullName = document.getElementById("fullName");
const email = document.getElementById("email");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
registerForm.addEventListener("submit", function (event) {
    let isValid = true;
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    nameError.textContent = "";
    emailError.textContent = "";
    if (fullName.value.trim() === "") {
        nameError.textContent = "Vui lòng nhập họ tên.";
        isValid = false;
    }
    if (email.value.trim() === "") {
        emailError.textContent = "Vui lòng nhập email.";
        isValid = false;
    } else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
        isValid = false;
    }
    if (!isValid) {
        event.preventDefault();
    }
});

```

Bảng sau đây tổng hợp kiểm tra dữ liệu cơ bản

Nội dung kiểm tra	Mục đích	Cách thực hiện cơ bản
Trường bắt buộc	Không cho phép để trống dữ liệu quan trọng	Kiểm tra value.trim() === ""
Email đơn giản	Bảo đảm email có dạng cơ bản hợp lệ	Dùng RegExp và test()

Nội dung kiểm tra	Mục đích	Cách thực hiện cơ bản
Chuỗi chỉ có khoảng trắng	Tránh nhập giả tạo	Dùng trim() trước khi kiểm tra
Kiểm tra trước khi gửi form	Ngăn dữ liệu sai được gửi đi	Kết hợp trong sự kiện submit

Bảng 4.16: Tổng hợp kiểm tra dữ liệu cơ bản

2.4.2. Hiện thị thông báo lỗi thân thiện với người dùng.

Kiểm tra dữ liệu chỉ thực sự hiệu quả khi người dùng hiểu mình đang sai ở đâu và cần sửa như thế nào. Vì vậy, thông báo lỗi không nên chỉ nói chung chung rằng “Dữ liệu không hợp lệ”, mà cần chỉ rõ trường nào sai và sai theo hướng nào.

a. Nguyên tắc hiện thị lỗi thân thiện

Thông báo lỗi thân thiện nên đáp ứng các yêu cầu sau:

- xuất hiện gần trường bị lỗi,
- ngắn gọn, rõ nghĩa,
- dùng ngôn ngữ dễ hiểu,
- chỉ ra cách người dùng cần sửa,
- không làm người dùng cảm thấy bị “báo lỗi kỹ thuật”.

Ví dụ tốt:

- “Vui lòng nhập họ tên.”
- “Email chưa đúng định dạng.”
- “Mật khẩu cần ít nhất 6 ký tự.”

Ví dụ chưa tốt:

- “Input invalid.”
- “Lỗi dữ liệu.”
- “Form submit failed.”

b. Cách tổ chức vùng hiện thị lỗi trong HTML

Mỗi trường nhập nên có một vùng nhỏ đi kèm để hiện thị lỗi. Cách này rõ ràng hơn việc dùng một thông báo chung cho toàn bộ form. Ví dụ

```
<div class="form-group">
  <label for="email">Email</label>
  <input type="text" id="email">
  <small id="emailError" class="error-message"></small>
</div>
```

c. CSS cơ bản cho thông báo lỗi

```
.error-message {
  color: red;
  font-size: 14px;
  display: block;
  margin-top: 4px;
}
.input-error {
  border: 1px solid red;
}
```

d. Hiển thị lỗi kết hợp đổi trạng thái giao diện

Ngoài việc in thông báo, chúng ta có thể thêm class CSS để làm nổi bật trường đang sai.

```
if (email.value.trim() === "") {
  emailError.textContent = "Vui lòng nhập email.";
  email.classList.add("input-error");
}
```

Khi trường hợp lệ trở lại, cần xóa class lỗi để giao diện phản ánh đúng trạng thái mới.

```
emailError.textContent = "";
email.classList.remove("input-error");
```

Ví dụ hoàn chỉnh: thông báo lỗi thân thiện

```
<form id="contactForm">
  <div class="form-group">
    <label for="fullName">Họ tên</label>
    <input type="text" id="fullName">
    <small id="nameError" class="error-message"></small>
  </div>
  <div class="form-group">
    <label for="email">Email</label>
    <input type="text" id="email">
    <small id="emailError" class="error-message"></small>
  </div>
  <button type="submit">Gửi</button>
</form>
```

```
.form-group {
  margin-bottom: 12px;
}
.error-message {
  color: red;
  font-size: 14px;
  display: block;
  min-height: 18px;
}
.input-error {
  border: 1px solid red;
}
const contactForm = document.getElementById("contactForm");
const fullName = document.getElementById("fullName");
const email = document.getElementById("email");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
contactForm.addEventListener("submit", function (event) {
  let isValid = true;
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  nameError.textContent = "";
  emailError.textContent = "";
  fullName.classList.remove("input-error");
  email.classList.remove("input-error");
  if (fullName.value.trim() === "") {
    nameError.textContent = "Vui lòng nhập họ tên.";
    fullName.classList.add("input-error");
    isValid = false;
  }
  if (email.value.trim() === "") {
    emailError.textContent = "Vui lòng nhập email.";
    email.classList.add("input-error");
    isValid = false;
  } else if (!emailPattern.test(email.value.trim())) {
    emailError.textContent = "Email chưa đúng định dạng, ví dụ tenban@example.com.";
  }
});
```

```

        email.classList.add("input-error");
        isValid = false;
    }
    if (!isValid) {
        event.preventDefault();
    }
});

```

Phân tích kỹ thuật

- Mỗi lần submit, chương trình xóa toàn bộ lỗi cũ trước.
- Sau đó kiểm tra lại từng trường.
- Nếu có lỗi, hiển thị thông báo ngay dưới trường tương ứng.
- Đồng thời thêm class CSS để làm nổi bật ô nhập sai.
- Cách tổ chức này giúp giao diện phản hồi rõ ràng, không gây nhầm lẫn cho người dùng.

Bảng sau tổng hợp các nguyên tắc thông báo lỗi thân thiện

Nguyên tắc	Ý nghĩa
Đặt lỗi gần trường nhập	Người dùng dễ nhận biết lỗi thuộc về đâu
Nội dung ngắn, rõ	Dễ đọc, dễ sửa
Chỉ ra hướng sửa	Hỗ trợ thao tác tiếp theo
Không dùng thông báo kỹ thuật khó hiểu	Phù hợp với người dùng phổ thông
Kết hợp thay đổi giao diện trường lỗi	Tăng khả năng nhận diện trực quan

Bảng 4.14: Nguyên tắc thông báo lỗi thân thiện

2.4.3. Ngăn gửi form khi dữ liệu không hợp lệ.

Đây là bước hoàn thiện luồng kiểm tra dữ liệu. Nếu phát hiện dữ liệu chưa đúng mà chương trình vẫn cho phép gửi form, quá trình xác thực sẽ mất ý nghĩa. Vì vậy, trong sự kiện submit, chúng ta cần chặn hành vi gửi mặc định khi có lỗi.

a. Cơ chế ngăn gửi form

Khi người dùng nhấn nút submit, trình duyệt sẽ phát sinh sự kiện submit. Trong hàm xử lý, nếu thấy dữ liệu không hợp lệ, chúng ta dùng:

```
event.preventDefault();
```

Lệnh này ngăn trình duyệt gửi form đi.

b. Mô hình kiểm tra hợp lệ bằng cờ isValid

Cách triển khai phổ biến và dễ dạy là dùng một biến logic, thường đặt tên là isValid.

- Ban đầu giả sử form hợp lệ: isValid = true.
 - Mỗi khi phát hiện lỗi, đặt isValid = false.
 - Sau khi kiểm tra xong, nếu isValid là false thì gọi event.preventDefault().
- Cách làm này rõ ràng, dễ mở rộng khi có nhiều trường dữ liệu.

Ví dụ hoàn chỉnh: ngăn gửi form khi sai

```
<form id="signupForm">
  <div class="form-group">
    <label for="fullName">Họ tên</label>
    <input type="text" id="fullName">
    <small id="nameError" class="error-message"></small>
  </div>
  <div class="form-group">
    <label for="email">Email</label>
    <input type="text" id="email">
    <small id="emailError" class="error-message"></small>
  </div>
  <button type="submit">Đăng ký</button>
</form>
<p id="successMessage"></p>
.form-group {
  margin-bottom: 12px;
}
.error-message {
  color: red;
  display: block;
  min-height: 18px;
}
.input-error {
  border: 1px solid red;
}
#successMessage {
  color: green;
```

```
margin-top: 10px;
}
const signupForm = document.getElementById("signupForm");
const fullName = document.getElementById("fullName");
const email = document.getElementById("email");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const successMessage =
document.getElementById("successMessage");
signupForm.addEventListener("submit", function (event) {
    let isValid = true;
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    nameError.textContent = "";
    emailError.textContent = "";
    successMessage.textContent = "";
    fullName.classList.remove("input-error");
    email.classList.remove("input-error");
    if (fullName.value.trim() === "") {
        nameError.textContent = "Họ tên không được để trống.";
        fullName.classList.add("input-error");
        isValid = false;
    }
    if (email.value.trim() === "") {
        emailError.textContent = "Email không được để trống.";
        email.classList.add("input-error");
        isValid = false;
    }
    else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
        email.classList.add("input-error");
        isValid = false;
    }
    if (!isValid) {
        event.preventDefault();
    }
    else {
        event.preventDefault();
        successMessage.textContent = "Dữ liệu hợp lệ. Form đã
```

```
sẵn sàng để gửi.";
    }
});
```

d. Giải thích luồng hoạt động

- Người dùng nhấn nút đăng ký.
- Sự kiện submit được kích hoạt.
- Chương trình xóa lỗi cũ và đưa giao diện về trạng thái kiểm tra mới.
- Từng trường được kiểm tra theo thứ tự.
- Nếu có lỗi, isValid chuyển thành false.
- Nếu isValid là false, form bị chặn gửi.
- Nếu dữ liệu đúng, ở ví dụ học tập này chúng ta vẫn preventDefault() để ở lại trang và hiển thị thông báo thành công. Trong hệ thống thực tế, lúc đó có thể cho form gửi đi hoặc dùng AJAX để xử lý.

e. Kiểm tra theo thời gian thực kết hợp kiểm tra lúc submit

Trong thực tế, có hai thời điểm thường được kiểm tra:

- kiểm tra ngay khi người dùng đang nhập hoặc rời khỏi ô dữ liệu,
- kiểm tra tổng thể khi người dùng submit form.

Ở giai đoạn nhập môn, nên cho sinh viên thấy rằng kiểm tra lúc submit là nền tảng, còn kiểm tra theo thời gian thực là lớp hỗ trợ trải nghiệm người dùng tốt hơn.

Ví dụ kiểm tra email khi người dùng gõ:

```
email.addEventListener("keyup", function () {
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (email.value.trim() === "") {
        emailError.textContent = "Vui lòng nhập email.";
    } else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
    } else {
        emailError.textContent = "";
    }
});
```

Bảng quy trình kiểm tra form hợp lệ

Bước	Nội dung xử lý	Mục tiêu
1	Bắt sự kiện submit	Kiểm soát thời điểm gửi form
2	Xóa thông báo lỗi cũ	Tránh hiển thị chồng lỗi
3	Kiểm tra từng trường	Xác định trường nào sai
4	Hiển thị lỗi tương ứng	Hướng dẫn người dùng sửa
5	Đặt isValid = false nếu có lỗi	Đánh dấu form không hợp lệ
6	Gọi event.preventDefault() khi cần	Ngăn form gửi đi
7	Xử lý thành công nếu hợp lệ	Cho phép gửi hoặc thông báo hoàn tất

Bảng 4.15: Quy trình kiểm tra form hợp lệ

Kế tiếp, chúng ta sẽ thực hiện một số bài thực hành để làm quen các khái niệm đã tìm hiểu trên.

Bài thực hành 1: Kiểm tra họ tên và email khi submit

Bước 1. Tạo file index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kiểm tra form cơ bản</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 500px;
      margin: 20px auto;
    }
    .form-group {
      margin-bottom: 14px;
    }
  </style>
</head>
<body>
  <div class="form-group">
    <input type="text" value="Họ tên" />
  </div>
  <div class="form-group">
    <input type="text" value="Email" />
  </div>
  <div class="form-group">
    <input type="button" value="Gửi" />
  </div>
</body>
</html>
```

```

    label {
        display: block;
        margin-bottom: 4px;
    }
    input {
        width: 100%;
        padding: 8px;
        box-sizing: border-box;
    }
    .error-message {
        color: red;
        font-size: 14px;
        min-height: 18px;
        display: block;
        margin-top: 4px;
    }
    .input-error {
        border: 1px solid red;
    }
    #successMessage {
        color: green;
        margin-top: 10px;
    }
}
</style>
</head>
<body>
    <h1>Form đăng ký</h1>
    <form id="signupForm">
        <div class="form-group">
            <label for="fullName">Họ tên</label>
            <input type="text" id="fullName" placeholder="Nhập
họ tên">
            <small id="nameError" class="error-
message"></small>
        </div>

```

```

        <div class="form-group">
            <label for="email">Email</label>
            <input type="text" id="email" placeholder="Nhập
email">
            <small id="emailError" class="error-
message"></small>
        </div>
        <button type="submit">Đăng ký</button>
    </form>
    <p id="successMessage"></p>
    <script>
        const signupForm =
document.getElementById("signupForm");
        const fullName = document.getElementById("fullName");
        const email = document.getElementById("email");
        const nameError = document.getElementById("nameError");
        const emailError =
document.getElementById("emailError");
        const successMessage =
document.getElementById("successMessage");
        signupForm.addEventListener("submit", function (event)
{
            let isValid = true;
            const emailPattern =
/^([^\s@]+@[^\s@]+\.[^\s@]+)$/;
            nameError.textContent = "";
            emailError.textContent = "";
            successMessage.textContent = "";
            fullName.classList.remove("input-error");
            email.classList.remove("input-error");
            if (fullName.value.trim() === "") {
                nameError.textContent = "Vui lòng nhập họ
tên.";
                fullName.classList.add("input-error");
                isValid = false;
            }
        }
    </script>

```

```

        if (email.value.trim() === "") {
            emailError.textContent = "Vui lòng nhập
email.";

            email.classList.add("input-error");
            isValid = false;
        } else if (!emailPattern.test(email.value.trim()))
{
            emailError.textContent = "Email chưa đúng định
dạng.";

            email.classList.add("input-error");
            isValid = false;
        }
        if (!isValid) {
            event.preventDefault();
        } else {
            event.preventDefault();
            successMessage.textContent = "Form hợp lệ, sẵn
sàng gửi dữ liệu.";
        }
    });
</script>
</body>
</html>

```

Bước 2. Mở file trên trình duyệt hoặc dùng Live Server

Bước 3. Thử các tình huống

- để trống cả hai trường rồi bấm đăng ký,
- chỉ nhập họ tên,
- nhập email sai định dạng như abc@,
- nhập đúng cả hai trường.

Bước 4. Quan sát kết quả

- lỗi hiển thị gần từng trường,
- ô sai được viền đỏ,
- dữ liệu không hợp lệ thì form không gửi,
- dữ liệu hợp lệ thì hiển thị thông báo thành công.

Bài thực hành 2: Kiểm tra theo thời gian thực

Ở bài này, chúng ta tăng thêm trải nghiệm người dùng bằng cách kiểm tra email trong lúc nhập.

```
<input type="text" id="email" placeholder="Nhập email">
<small id="emailError" class="error-message"></small>
const email = document.getElementById("email");
const emailError = document.getElementById("emailError");
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
email.addEventListener("keyup", function () {
    if (email.value.trim() === "") {
        emailError.textContent = "Vui lòng nhập email.";
    } else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
    } else {
        emailError.textContent = "";
    }
});
```

Kết quả kỳ vọng:

- Người dùng không cần đợi đến lúc submit mới biết mình sai.
- Tuy nhiên vẫn cần kiểm tra lại ở bước submit vì người dùng có thể bỏ qua một số tình huống.

Lỗi thường gặp khi học kiểm tra form được tổng hợp qua bảng sau:

Lỗi	Nguyên nhân	Cách khắc phục
Form vẫn gửi dù có lỗi	Quên <code>event.preventDefault()</code>	Gọi chặn submit khi dữ liệu sai
Dữ liệu chỉ gồm khoảng trắng vẫn được chấp nhận	Không dùng <code>trim()</code>	Luôn kiểm tra với <code>value.trim()</code>
Email hợp lệ nhưng vẫn báo sai	Biểu thức kiểm tra viết sai	Kiểm tra lại RegExp
Thông báo lỗi không biến mất	Không xóa lỗi cũ trước khi kiểm tra lại	Reset trạng thái ở đầu hàm

Lỗi	Nguyên nhân	Cách khắc phục
Ô nhập vẫn đỏ dù đã sửa đúng	Quên xóa class lỗi	Dùng <code>classList.remove()</code> trước khi kiểm tra mới

Bảng 4.16: Lỗi thường gặp khi học kiểm tra form

2.5. Thực hành tổng hợp bài 3

Mục thực hành tổng hợp này có vai trò kết nối toàn bộ kiến thức của Bài 3 thành một sản phẩm web có tương tác thực sự. Nếu ở các mục trước chúng ta lần lượt học về JavaScript cơ bản, DOM, sự kiện và kiểm tra dữ liệu biểu mẫu, thì ở đây các nội dung đó sẽ được vận dụng đồng thời trong một website hoàn chỉnh đã được xây dựng từ các bài trước.

Mục tiêu của phần thực hành tổng hợp gồm ba lớp. Lớp thứ nhất là gắn JavaScript vào website đã có để kích hoạt tương tác. Lớp thứ hai là triển khai các tính năng thiết yếu như menu động, kiểm tra form và hiệu ứng ẩn hiện nội dung. Lớp thứ ba là kiểm thử trên nhiều trình duyệt để phát hiện lỗi hiển thị, lỗi tương tác và tinh chỉnh cho website hoạt động ổn định hơn.

2.5.1. Gắn JavaScript tương tác cho website đã xây dựng ở các bài trước.

a. Ý nghĩa của bước tích hợp

Ở các bài trước, website của chúng ta chủ yếu mới dừng ở mức giao diện tĩnh, nghĩa là người dùng có thể nhìn thấy nội dung nhưng chưa có nhiều phản hồi thông minh khi thao tác. Việc gắn JavaScript tương tác chính là bước biến website từ trạng thái tĩnh sang trạng thái động.

Một website học tập, giới thiệu cá nhân, lớp học hay cửa hàng nhỏ thường cần tối thiểu các khả năng sau:

- phản hồi khi người dùng nhấn menu hoặc nút bấm,
- mở hoặc thu gọn một vùng nội dung,
- kiểm tra dữ liệu form liên hệ hoặc đăng ký,
- thay đổi trạng thái giao diện dựa trên hành động của người dùng.

b. Tổ chức mã nguồn trong bài thực hành tổng hợp

Để thực hành hiệu quả trên VS Code, chúng ta nên tách mã thành ba phần rõ ràng như mô tả từ bảng sau:

Tệp	Vai trò	Nội dung chính
index.html	Cấu trúc giao diện	header, menu, nội dung, form
style.css	Trình bày giao diện	bố cục, màu sắc, trạng thái ẩn/hiện
script.js	Xử lý tương tác	menu động, form, hiệu ứng DOM

Bảng 4.17: Tổ chức mã nguồn trong bài thực hành tổng hợp

Cách tổ chức này giúp sinh viên thấy rõ nguyên tắc tách biệt:

- HTML mô tả nội dung và cấu trúc,
- CSS điều khiển cách hiển thị,
- JavaScript điều khiển hành vi.

c. Mẫu website tích hợp thực hành

Trong phần này, chúng ta xây dựng một website nhỏ gồm:

- thanh menu,
- phần giới thiệu,
- nút ẩn hiện nội dung,
- biểu mẫu liên hệ.

File index.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Website thực hành JavaScript</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <h1>Website thực hành tổng hợp</h1>
    <button id="menuToggle" class="menu-toggle">≡
Menu</button>
    <nav id="mainMenu" class="main-menu">
      <a href="#home">Trang chủ</a>
      <a href="#about">Giới thiệu</a>
```

```

        <a href="#contact">Liên hệ</a>
    </nav>
</header>
<main class="container">
    <section id="home">
        <h2>Trang chủ</h2>
        <p>Đây là website được bổ sung tương tác bằng
JavaScript.</p>
    </section>
    <section id="about">
        <h2>Giới thiệu</h2>
        <button id="toggleAboutBtn">Ẩn/Hiện nội dung giới
thiệu</button>
        <div id="aboutContent" class="content-box">
            <p>
                Nội dung này có thể được ẩn hoặc hiện bằng
JavaScript để minh họa thao tác DOM và xử lý sự kiện.
            </p>
        </div>
    </section>
    <section id="contact">
        <h2>Liên hệ</h2>
        <form id="contactForm" novalidate>
            <div class="form-group">
                <label for="fullName">Họ tên</label>
                <input type="text" id="fullName"
placeholder="Nhập họ tên">
                <small id="nameError" class="error-
message"></small>
            </div>
            <div class="form-group">
                <label for="email">Email</label>
                <input type="text" id="email"
placeholder="Nhập email">
                <small id="emailError" class="error-
message"></small>

```

```
        </div>
        <button type="submit">Gửi liên hệ</button>
    </form>
    <p id="successMessage" class="success-message"></p>
</section>
</main>
<script src="script.js"></script>
</body>
</html>
```

File style.css

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    line-height: 1.6;
    background: #f5f5f5;
    color: #222;
}
.site-header {
    background: #1f4e79;
    color: white;
    padding: 16px;
}
.site-header h1 {
    margin: 0 0 10px;
    font-size: 24px;
}
.menu-toggle {
    padding: 8px 12px;
    border: none;
    cursor: pointer;
}
.main-menu {
    display: none;
    margin-top: 12px;
}
.main-menu.show {
```

```
    display: block;
}
.main-menu a {
    display: block;
    color: white;
    text-decoration: none;
    padding: 6px 0;
}
.container {
    max-width: 900px;
    margin: 20px auto;
    background: white;
    padding: 20px;
}
.content-box {
    margin-top: 10px;
    padding: 12px;
    border: 1px solid #ccc;
    background: #fafafa;
}
.hidden {
    display: none;
}
.form-group {
    margin-bottom: 14px;
}
label {
    display: block;
    margin-bottom: 4px;
    font-weight: bold;
}
input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
}
```

```

.error-message {
  color: red;
  font-size: 14px;
  display: block;
  min-height: 18px;
  margin-top: 4px;
}
.input-error {
  border: 1px solid red;
}
.success-message {
  color: green;
  margin-top: 12px;
}
@media (min-width: 768px) {
  .menu-toggle {
    display: none;
  }
  .main-menu {
    display: flex;
    gap: 20px;
  }
  .main-menu a {
    display: inline-block;
    padding: 0;
  }
}

```

File script.js

```

const menuToggle = document.getElementById("menuToggle");
const mainMenu = document.getElementById("mainMenu");
const toggleAboutBtn =
document.getElementById("toggleAboutBtn");
const aboutContent = document.getElementById("aboutContent");
const contactForm = document.getElementById("contactForm");
const fullName = document.getElementById("fullName");
const email = document.getElementById("email");

```

```
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const successMessage =
document.getElementById("successMessage");
menuToggle.addEventListener("click", function () {
    mainMenu.classList.toggle("show");
});
toggleAboutBtn.addEventListener("click", function () {
    aboutContent.classList.toggle("hidden");
});
contactForm.addEventListener("submit", function (event) {
    event.preventDefault();
    let isValid = true;
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    nameError.textContent = "";
    emailError.textContent = "";
    successMessage.textContent = "";
    fullName.classList.remove("input-error");
    email.classList.remove("input-error");
    if (fullName.value.trim() === "") {
        nameError.textContent = "Vui lòng nhập họ tên.";
        fullName.classList.add("input-error");
        isValid = false;
    }
    if (email.value.trim() === "") {
        emailError.textContent = "Vui lòng nhập email.";
        email.classList.add("input-error");
        isValid = false;
    } else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
        email.classList.add("input-error");
        isValid = false;
    }
    if (isValid) {
        successMessage.textContent = "Biểu mẫu hợp lệ. Dữ liệu
đã sẵn sàng để gửi.";
```

```
        contactForm.reset();
    }
});
```

Trong ví dụ trên, chúng ta đã tích hợp ba vùng tương tác chính vào cùng một website. Thứ nhất là menu động. Nút menuToggle dùng sự kiện click để thêm hoặc xóa class show cho khối menu. Nhờ CSS, menu sẽ được hiện hoặc ẩn theo trạng thái đó. Thứ hai là vùng giới thiệu có thể ẩn hoặc hiện. Khi nhấn nút, class hidden được bật hoặc tắt trên phần nội dung. Đây là ứng dụng trực tiếp của DOM và `classList.toggle()`. Thứ ba là form liên hệ. Khi người dùng submit, JavaScript kiểm tra dữ liệu đầu vào, hiển thị lỗi nếu cần và chỉ báo thành công khi dữ liệu đạt yêu cầu.

Quy trình thực hành trên VS Code

Bước 1. Tạo thư mục dự án

Tạo một thư mục ví dụ tên `bai3-thuc-hanh-tong-hop`.

Bước 2. Tạo ba tệp chính

- `index.html`
- `style.css`
- `script.js`

Bước 3. Dán mã nguồn tương ứng vào từng tệp

Bước 4. Mở `index.html` bằng trình duyệt hoặc Live Server

Bước 5. Kiểm tra lần lượt từng chức năng

- nhấn nút menu trên màn hình hẹp,
- nhấn nút ẩn hiện phần giới thiệu,
- nhập sai rồi nhập đúng trong biểu mẫu liên hệ.

Phần thực hành này giúp người học vượt qua giai đoạn học từng kỹ thuật riêng lẻ. Thay vì nghĩ “DOM là một mảng kiến thức” và “form validation là một mảng khác”, người học bắt đầu thấy chúng phối hợp thành sản phẩm cụ thể như thế nào. Đây là bước rất quan trọng trước khi học các dự án giao diện lớn hơn.

2.5.2. Thêm các tính năng: menu động, kiểm tra form, hiệu ứng ẩn/hiện nội dung.

Ở mục này, chúng ta phân tích sâu hơn ba nhóm tính năng đã tích hợp trong website để sinh viên hiểu không chỉ cách làm mà còn hiểu vì sao phải làm như vậy.

a. Menu động

Menu động thường được dùng trong giao diện responsive, đặc biệt trên màn hình nhỏ. Thay vì hiển thị toàn bộ các liên kết điều hướng theo chiều ngang, website sẽ thu gọn menu và chỉ mở ra khi người dùng nhấn nút.

Menu động trong ví dụ trên được xây dựng theo mô hình rất cơ bản:

- HTML có một nút điều khiển và một vùng menu,
- CSS ẩn menu ở trạng thái mặc định trên màn hình nhỏ,
- JavaScript gắn sự kiện click cho nút,
- khi nhấn, JavaScript thêm hoặc xóa class show.

Mã sử dụng:

```
menuToggle.addEventListener("click", function () {  
    mainMenu.classList.toggle("show");  
});
```

Giải thích

- menuToggle là nút bấm.
- mainMenu là vùng chứa các liên kết.
- toggle("show") đổi trạng thái hiển thị.
- Cách làm này đơn giản nhưng thể hiện đúng mô hình xử lý tương tác thực tế.

b. Kiểm tra form

Form liên hệ hoặc đăng ký là một trong những thành phần dễ phát sinh lỗi nhất vì dữ liệu đến từ người dùng. Do đó, kiểm tra form giúp website:

- không nhận dữ liệu rỗng,
- hạn chế sai định dạng,
- phản hồi rõ ràng để người dùng sửa.

Luồng kiểm tra form cơ bản theo bảng sau:

Bước	Hành động
1	Bắt sự kiện submit
2	Ngăn gửi mặc định để JavaScript xử lý
3	Xóa thông báo lỗi cũ
4	Kiểm tra từng trường
5	Hiển thị lỗi nếu có

Bước	Hành động
6	Nếu hợp lệ thì thông báo thành công hoặc tiếp tục gửi

Bảng 4.18: Luồng kiểm tra cơ bản của form

Mã sử dụng

```
contactForm.addEventListener("submit", function (event) {
    event.preventDefault();
    let isValid = true;
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    nameError.textContent = "";
    emailError.textContent = "";
    if (fullName.value.trim() === "") {
        nameError.textContent = "Vui lòng nhập họ tên.";
        isValid = false;
    }
    if (email.value.trim() === "") {
        emailError.textContent = "Vui lòng nhập email.";
        isValid = false;
    } else if (!emailPattern.test(email.value.trim())) {
        emailError.textContent = "Email chưa đúng định dạng.";
        isValid = false;
    }
});
```

c. Hiệu ứng ẩn/hiện nội dung

Hiệu ứng ẩn hiện giúp giao diện gọn hơn, giảm lượng nội dung phải hiển thị cùng lúc và tạo cảm giác có phản hồi khi người dùng thao tác. Cơ chế hoạt động

- HTML có một nút và một khối nội dung.
- CSS định nghĩa class .hidden { display: none; }.
- JavaScript gắn sự kiện click cho nút để bật hoặc tắt class hidden.

Mã sử dụng

```
toggleAboutBtn.addEventListener("click", function () {
    aboutContent.classList.toggle("hidden");
});
```

2.5.3. Kiểm thử trên nhiều trình duyệt, chỉnh sửa để giao diện và chức năng hoạt động ổn định

Sau khi hoàn thành các chức năng, website chưa thể xem là hoàn chỉnh nếu chưa được kiểm thử. Trong thực tế, nhiều lỗi chỉ xuất hiện khi thay đổi trình duyệt, kích thước màn hình hoặc cách thao tác của người dùng. Vì vậy, kiểm thử là bước bắt buộc để bảo đảm tính ổn định.

a. Mục tiêu của kiểm thử

Kiểm thử trong bài thực hành tổng hợp nhằm xác nhận ba nhóm vấn đề:

- giao diện có hiển thị đúng không,
- chức năng có hoạt động đúng không,
- trải nghiệm sử dụng có nhất quán không.

b. Kiểm thử trên nhiều trình duyệt

Ở mức cơ bản, nên thử website trên ít nhất các môi trường sau:

Trình duyệt / môi trường	Nội dung cần kiểm tra
Google Chrome	hiển thị tổng thể, menu, form, console lỗi
Microsoft Edge	tính tương thích giao diện và sự kiện
Firefox	hành vi DOM, CSS, form validation
Màn hình điện thoại hoặc chế độ responsive	menu thu gọn, bố cục form, nút bấm

Bảng 4.20: Kiểm thử trên nhiều trình duyệt

Quy trình kiểm thử đề xuất

Bước	Nội dung kiểm thử	Kết quả mong đợi
1	Mở website trên Chrome	Trang tải đúng, không lỗi console
2	Thu nhỏ màn hình	Menu chuyển sang dạng thu gọn
3	Nhấn nút menu	Menu hiện và ẩn đúng
4	Nhấn nút ẩn/hiện giới thiệu	Nội dung thay đổi trạng thái đúng
5	Submit form rỗng	Hiện lỗi phù hợp, không gửi form
6	Submit email sai định dạng	Báo lỗi đúng trường email
7	Submit dữ liệu đúng	Hiện thông báo thành công

Bước	Nội dung kiểm thử	Kết quả mong đợi
8	Mở trên trình duyệt khác	Chức năng và giao diện không sai lệch lớn

Bảng 4.21: Quy trình kiểm thử đề xuất

Khi kiểm thử trên VS Code và trình duyệt, chúng ta nên hướng dẫn sinh viên sử dụng bộ công cụ Developer Tools.

Tổng kết

Bài 3 giúp người học bước đầu khai thác JavaScript như một ngôn ngữ tạo tương tác trên trình duyệt, bổ sung cho cấu trúc HTML và định dạng CSS đã học ở các bài trước.

Người học đã ôn lại các thành phần nền tảng của JavaScript như biến, kiểu dữ liệu, toán tử, câu lệnh rẽ nhánh và lặp; đồng thời nhận biết được các cách chèn mã JavaScript vào trang HTML theo hình thức trực tiếp, nội bộ và tệp ngoài. Trọng tâm của bài là làm việc với DOM: chọn phần tử, thay đổi nội dung, thuộc tính, lớp CSS, cũng như tạo, chèn, xóa phần tử để giao diện phản hồi theo thao tác của người dùng. Bài học cũng rèn luyện kỹ năng xử lý sự kiện như click, change, keyup, submit; từ đó xây dựng được một số thành phần tương tác cơ bản như nút ẩn/hiện nội dung, menu động, tab hoặc accordion.

Ngoài ra, người học đã biết cách kiểm tra dữ liệu nhập trên biểu mẫu ở mức cơ bản, hiển thị thông báo lỗi thân thiện và ngăn gửi form khi dữ liệu chưa hợp lệ. Đây là bước đệm quan trọng trước khi chuyển sang tư duy phát triển giao diện theo component ở Bài 4 - tiếp cận framework giao diện, nơi các thành phần được tổ chức theo mô hình component và dễ mở rộng hơn trong các ứng dụng hiện đại.

Bài tập và đánh giá

A. Câu hỏi lý thuyết (5 điểm)

Câu 1 (1 điểm) – Vai trò của JavaScript trong trang web

Trình bày vai trò của JavaScript trong thiết kế web phía client. Phân biệt ngắn gọn nhiệm vụ chính của HTML, CSS và JavaScript trong một trang web hiện đại.

Câu 2 (1 điểm) – DOM và thao tác với phần tử

Giải thích khái niệm DOM. Nêu quy trình cơ bản để chọn một phần tử HTML và thay đổi nội dung hoặc thuộc tính của phần tử đó bằng JavaScript.

Câu 3 (1 điểm) – Xử lý sự kiện người dùng

Trình bày khái niệm sự kiện trong JavaScript trên trình duyệt. Cho ví dụ về ít nhất hai sự kiện thường gặp và nêu cách gắn hàm xử lý cho phần tử.

Câu 4 (1 điểm) – Kiểm tra dữ liệu nhập trên form

Nêu mục đích của việc kiểm tra dữ liệu nhập phía client. Trình bày một số nội dung cần kiểm tra trong biểu mẫu như trường bắt buộc, định dạng email hoặc độ dài dữ liệu.

Câu 5 (1 điểm) – Tổ chức mã JavaScript cơ bản

So sánh ba cách chèn JavaScript vào trang HTML: inline, internal và external. Theo anh/chị, trong quá trình học và thực hành nên ưu tiên cách nào để mã nguồn dễ quản lý hơn?

B. Câu hỏi thực hành (5 điểm)

Thực hành 1 (2,5 điểm)

Yêu cầu: Tạo một trang HTML đơn giản có nút bấm thay đổi nội dung của một đoạn văn hoặc tiêu đề. Khi người dùng nhấn nút, nội dung văn bản phải thay đổi rõ ràng; đồng thời màu chữ hoặc lớp CSS của phần tử cũng được cập nhật để thể hiện trạng thái mới.

Nội dung cần đạt: liên kết đúng tệp JavaScript; chọn đúng phần tử bằng id hoặc selector phù hợp; xử lý sự kiện click; cập nhật nội dung và giao diện trực quan sau thao tác.

Thực hành 2 (2,5 điểm)

Yêu cầu: Xây dựng một biểu mẫu liên hệ hoặc đăng ký ngắn gồm họ tên, email và nội dung nhập liệu. Dùng JavaScript để kiểm tra dữ liệu trước khi gửi: không để trống trường bắt buộc, kiểm tra định dạng email ở mức đơn giản và hiển thị thông báo lỗi gần vị trí nhập.

Nội dung cần đạt: bắt được sự kiện submit; sử dụng điều kiện kiểm tra hợp lý; thông báo lỗi rõ ràng, dễ hiểu; ngăn gửi biểu mẫu khi dữ liệu không hợp lệ; cho phép gửi khi dữ liệu đạt yêu cầu.

C. Bảng đánh giá (thang 10 điểm)

C.1. Phân bố điểm

Nhóm	Nội dung	Điểm
Lý thuyết	Câu 1 – Câu 5	5,0
Thực hành	Thực hành 1	2,5

Thực hành	Thực hành 2	2,5
Tổng		10,0

C.2. Chấm câu lý thuyết (mỗi câu tối đa 1 điểm)

Mức	Điểm	Tiêu chí	Gợi ý lỗi thường gặp
Đạt tối đa	1,0	Trả lời đúng trọng tâm; nêu rõ vai trò của JavaScript, DOM, sự kiện, kiểm tra form và cách chèn mã; thuật ngữ cơ bản chính xác; có ví dụ phù hợp với ngữ cảnh trang web.	Nhầm lẫn giữa HTML, CSS và JavaScript; mô tả mơ hồ về DOM; nêu ví dụ không đúng loại sự kiện hoặc kiểm tra dữ liệu chưa sát yêu cầu.
Đạt một phần	0,5	Nêu được ý chính nhưng thiếu bước thực hiện, thiếu ví dụ hoặc diễn đạt chưa rõ mối liên hệ giữa thao tác JavaScript và giao diện web.	Chỉ kể khái niệm chung; chưa phân biệt inline/internal/external; chưa chỉ ra tác dụng của validate phía client.
Chưa đạt	0,0	Sai trọng tâm hoặc bỏ trống phần lớn nội dung.	Không nêu được kiến thức cốt lõi của bài.

C.3. Chấm Thực hành 1 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Minh chứng cần có
Liên kết và tổ chức tệp	0,5	Tệp HTML và JS liên kết đúng; mã nguồn rõ	Có liên kết nhưng còn lỗi nhỏ về tổ chức hoặc	Tệp .html, tệp .js, ảnh chụp kết quả hoặc video thao tác.

		ràng, dễ đọc.	đường dẫn.	
Chọn phần tử và xử lý sự kiện	1,0	Chọn đúng phần tử; xử lý click đúng logic; thao tác chạy ổn định.	Có xử lý sự kiện nhưng chưa chính xác hoàn toàn hoặc còn lỗi nhỏ.	Đoạn mã JavaScript thể hiện selector và addEventListener/onclick.
Thay đổi nội dung và giao diện	1,0	Nội dung thay đổi rõ; có cập nhật màu sắc/lớp CSS; kết quả trực quan.	Chỉ thay đổi được một phần hoặc hiệu ứng còn chưa rõ ràng.	Ảnh trước và sau khi nhấn nút; mã nguồn cập nhật nội dung/CSS.

C.4. Chấm Thực hành 2 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Minh chứng cần có
Cấu trúc biểu mẫu	0,5	Form có đủ trường cơ bản, nhãn rõ ràng, bố cục dễ nhập.	Thiếu một trường hoặc bố cục chưa rõ ràng.	Ảnh giao diện form, mã HTML biểu mẫu.
Kiểm tra dữ liệu bằng JavaScript	1,0	Kiểm tra được trường bắt buộc và email; logic	Có kiểm tra nhưng chưa đầy đủ hoặc còn sai ở một	Đoạn mã validate, ảnh thử dữ liệu sai/đúng.

		điều kiện hợp lý.	số trường hợp.	
Thông báo lỗi và điều khiển gửi form	1,0	Hiển thị lỗi thân thiện; ngăn gửi khi sai; cho phép gửi khi đúng.	Thông báo lỗi còn khó hiểu hoặc chặn form chưa ổn định.	Ảnh hoặc video thao tác; mã xử lý submit và thông báo.